

Every question you have to write, answered in full

What is in here

36 questions and 48 worked answers, from 9 papers and practice sets: the examiner's own past papers, the mock papers, and every written question in the Learn by doing section. Each one is quoted as printed, broken down, then answered in full.

The objective half lives in the [question book](#).

Learn by Doing: Types, Operators, Input and Output

Topic Two

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question Five, 12 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Five

THE COURSE TEXT, UNEDITED

(12 marks)A. A conference organiser needs a program (using JOptionPane only, for all input/output) that prices tickets for a 3-day conference and produces a sales summary. Implement the following four methods.

- i. Create a method that takes in a day number. Use a switch statement to return a base ticket price for Day 1, Day 2, or Day 3 (pick any three distinct prices and state them clearly). If the day number is not 1, 2, or 3, throw an IllegalArgumentException.
- ii. Create a method that takes in a tier and a number of days attending. Use nested if-else statements (not a switch, and not independent/separate if checks) to return a discount percentage:
 - Gold, attending all 3 days -> 25%
 - Gold, attending 1-2 days -> 15%
 - Silver, attending all 3 days -> 15%

- Silver, attending 1-2 days -> 5%
- Bronze (any days) -> 0%
- Any tier string that is not "Gold" or "Silver" must be treated as "Bronze"

The nesting must reflect that "days attending" is only checked within each tier branch, not as a separate top-level check.

iii. Create a method that takes in a base price total and a discount percentage and applies the discount to the base price total. If the discounted amount is below 50.0, the method should return 50.0 instead (a price floor).

iv. `main()`

- Ask the user for the number of attendees N (a positive integer).
 - Loop exactly N times. In each iteration, ask for that attendee's number of days attending (an integer from 1 to 3, inclusive) and tier. For each day from 1 up to that attendee's days attending, call the method from part (i) and add the result to that attendee's running base price total (a running total that starts at 0 for each new attendee). Then call the method from part (ii) to get the discount percentage, and the method from part (iii) to get that attendee's final price.
 - Accumulate: (a) total revenue across all attendees, and (b) a running count of how many attendees were Gold, Silver, and Bronze.
- v. After the loop, show one summary dialog with total revenue and the three tier counts.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A complete pricing specification and the structure of the four methods.

Asked for	Four methods plus a main, with the control structure of each dictated : switch in part i, nested if-else in part ii.
The marks	12 marks. The structural constraints are marked, so a correct answer written the wrong way loses marks.
Watch	Part ii spells out the nesting it wants and then repeats it in a separate sentence, which is the examiner telling you where the marks are. Tier is the outer test, days the inner. Anything not "Gold" or "Silver" falls to Bronze, so that is the else branch, not a third named case.
Taught in	Units 4.2, 4.3, 5.4, 9.4 and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **12 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 A · CONFERENCE TICKET PRICING · 12 MARKS

1. takes a day number, uses a **switch** to return a base price for Day 1, 2 or 3 (**pick any three distinct prices and state them clearly**); any other day **throws IllegalArgumentException**;
2. takes a tier and days attending, and uses **nested if-else** to return a discount: Gold 3 days 25%, Gold 1-2 days 15%, Silver 3 days 15%, Silver 1-2 days 5%, Bronze 0%, and **any tier that is not "Gold" or "Silver" is Bronze**. The nesting must reflect that days are only checked **within** each tier branch;
3. takes a base total and a discount percentage, applies it, and returns **50.0** if the result would fall below 50.0 (a price floor);
4. **main()**: ask for N attendees; loop N times asking days and tier; sum the day prices into a per-attendee base total; get the discount; get the final price; accumulate total revenue and a count of Gold, Silver and Bronze; then show **one** summary dialog.

MUST MEMORISE · CHOOSE PRICES THAT LET THE FLOOR FIRE

The question lets you **pick the three prices**. That freedom is a trap with a right answer.

Pick 15000, 12000 and 9000 and part 3's price floor **can never fire**. The largest possible discount is 25%, so nothing ever falls near 50.0. You will have written a floor you cannot demonstrate and the examiner cannot see work.

Pick **30.0, 25.0 and 20.0** and the floor becomes live: one Gold day is 30, less 15% is 25.50, which is below 50.0, so the method returns 50.0. The 50.0 floor is itself the hint about the scale the examiner had in mind.

MEMORY HOOK: when a question lets you choose the numbers, choose numbers that make every rule visible. A rule that never fires is a rule you cannot be shown to have written correctly.

TEACH · WHAT "NESTED, NOT SEPARATE" ACTUALLY MEANS HERE

Part 2 is unusually explicit: nested if-else, **not** a switch, **not** independent if checks, and "the nesting must reflect that days attending is only checked **within** each tier branch, not as a separate top-level check."

So the shape is forced. The tier decides the branch; the days are examined **inside** it:

What the question demands	What loses the mark
<pre>if (Gold) { if (days == 3) 25 else 15 } else if (Silver) { if (days == 3) 15 else 5 } else { 0 }</pre>	<pre>if (Gold && days == 3) 25; if (Gold && days < 3) 15; if (Silver && days == 3) 15;</pre> Right answers, wrong structure: days is tested at the top level, four separate times. This is what "not independent checks" forbids.

Why the final else is the whole Bronze rule. "Any tier string that is not Gold or Silver must be treated as Bronze" is not an extra check you add: it is what a bare else already means. Write `else if (tier.equals("Bronze"))` and you have broken the rule, because "Platinum" would then match nothing and fall through with no discount assigned. The bare else catches Bronze, Platinum, "gold" in lower case, and an empty string, which is exactly what was asked (Unit 4.4).

The same else must also do the **counting**. If "Platinum" is treated as Bronze for the discount, it is a Bronze attendee in the summary too. Count it anywhere else and your dialog contradicts your own pricing.

CODE · QUESTION FIVE, PART A · VERIFIED ON THE JVM

```
import java.util.Scanner;public class ConferenceTickets {    // Part i: a switch returni
```

OutputAttendee 1: Gold, 3 day(s), base 75.0, discount 25.0%, pays 56.25

Attendee 2: Gold, 1 day(s), base 30.0, discount 15.0%, pays 50.0

Attendee 3: Silver, 2 day(s), base 55.0, discount 5.0%, pays 52.25

Attendee 4: Bronze, 1 day(s), base 30.0, discount 0.0%, pays 50.0

Attendee 5: Platinum, 3 day(s), base 75.0, discount 0.0%, pays 75.0

SUMMARY

Total revenue: 283.5

Gold: 2, Silver: 1, Bronze: 2

Read attendees 2 and 4: both would have paid less than 50 and both paid exactly 50.0, so **the floor is visibly working**. Read attendee 5: "Platinum" got a 0% discount **and** was counted as Bronze, so the two Bronze rules agree. Input is a Scanner here so the run can be printed; in the hall, every prompt is `JOptionPane.showInputDialog` and the summary is one `showMessageDialog` (F.11).

TRAP · THE SWITCH MUST RETURN, OR FALL THROUGH

Part 1's method must give back a price for 1, 2 and 3 and **throw** otherwise. Writing return inside each case is what makes that safe: a return leaves the method immediately, so no break is needed and there is no fall-through to worry about (Unit 4.3).

Use `double price`; with assignments instead and you must not forget the breaks, and the compiler will additionally complain that `price` "might not have been initialized" unless every path assigns it. The default: `throw` is what convinces the compiler every path is covered: an exception is a legitimate way to leave a method that promised to return a `double`, because it does not return at all.

Note also that `throw` needs no `throws` clause here. `IllegalArgumentException` is **unchecked** (Unit 10.1), which is Q1 B's distinction being examined a second time on the same paper.

Part 2: the method behind these, taught from nothing

Learn by Doing: Control Structures

Topic Three

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question One, 10 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question One

THE COURSE TEXT, UNEDITED

[10 Marks]D. An Armstrong number (also called a narcissistic number) is a positive integer that is equal to the sum of its own digits, each raised to the power of the total number of digits in the number. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

- i. State the five outputs for this code (you can show your workings but make the answer bold)
- ii. Retain other things but change the For Loop to While loop and the While loops to For loops.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A definition, a worked example, and a working program.

Asked **Two separate tasks:** trace five outputs, then rewrite every loop as the other kind. Part ii is for worth doing even if part i defeats you.

The marks 10 marks between them, the largest single block on the paper.

Watch Part ii says "**Retain other things**": change only the loop machinery, not the logic or the variable names. Note the swap is both ways, the for becomes a while *and* the two while loops become for loops.

Taught in Units 4.4 to 4.6 and 3.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **10 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 D I · STATE THE FIVE OUTPUTS · 10 MARKS, PART ONE

The answer. Compute each digit cubed (all five are 3-digit numbers, so the power is 3):

n	sum of digits ³	= n?	Output
370	27 + 343 + 0 = 370	yes	370 -> ARMSTRONG
371	27 + 343 + 1 = 371	yes	371 -> ARMSTRONG
372	27 + 343 + 8 = 378	no	372 -> NOT ARMSTRONG
407	64 + 0 + 343 = 407	yes	407 -> ARMSTRONG
408	64 + 0 + 512 = 576	no	408 -> NOT ARMSTRONG

The arithmetic: $3^3 = 27$, $7^3 = 343$, $0^3 = 0$, $1^3 = 1$, $2^3 = 8$, $4^3 = 64$, $8^3 = 512$. Three of the five are Armstrong numbers, which is worth noticing: the examiner chose 370, 371 and 407 because they are the three three-digit Armstrong numbers next to each other, and 372 and 408 as near misses.

CODE · CONFIRMED BY RUNNING THE PAPER'S OWN CODE · VERIFIED ON THE JVM

```
public class ArmstrongChecker {    public static void main(String[] args) {        int[]  
  
Output370 -> ARMSTRONG  
371 -> ARMSTRONG  
372 -> NOT ARMSTRONG  
407 -> ARMSTRONG  
408 -> NOT ARMSTRONG
```

Q1 D II · CONVERT THE LOOPS · 10 MARKS, PART TWO

Read it precisely. The original has **three** loops:

Loop	Currently	Becomes
the outer one over idx	for	while
counting the digits	while	for
summing the powers	while	for
the inner p loop	for	while

"The For Loop" (singular) is the outer one; "the While loops" (plural) are the two digit loops. The innermost p loop is also a for, so convert it too: the safest reading is that **every** for becomes a while and every while becomes a for. Apply Unit 4.6's recipe to each.

CODE · EVERY LOOP CONVERTED, SAME OUTPUT · VERIFIED ON THE JVM

```
public class ArmstrongConverted {    public static void main(String[] args) {        int
Output370 -> ARMSTRONG
371 -> ARMSTRONG
372 -> NOT ARMSTRONG
407 -> ARMSTRONG
408 -> NOT ARMSTRONG
```

Identical output to the original. That is the test of a conversion: same program, different clothes.

Two points a marker looks for. The for (; temp > 0; temp /= 10) has an **empty initialisation**, which is legal and is the honest conversion: the original while had nothing to initialise, because temp already existed. Do not invent a variable to fill the slot. And idx++ sits at the **bottom** of the while body, which is Unit 4.6's step 3 and the one people forget: leave it out and the program runs forever.

Question Two: 2025/2026, Question Two, 10 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Two

THE COURSE TEXT, UNEDITED

[10 marks]D. A perfect number is a positive integer that is equal to the sum of its own proper positive divisors (all divisors excluding the number itself). For example, 6 is perfect because $1 + 2 + 3 = 6$. A number whose proper divisors sum to more than the number itself is called abundant, and one whose proper divisors sum to less than the number itself is called deficient.

i. State the output for this code (no need to show workings)

ii. Change the For loops to While loops

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Three definitions and a working program.

Asked for The output, then a mechanical loop conversion.

The marks 10 marks. Part i explicitly says "**no need to show workings**", so do not spend ten minutes tabulating 30 numbers.

Watch Both loops convert this time, including the inner one, and the inner loop's bound depends on the outer variable. Converting the outer loop while leaving the inner is the usual half-answer.

Taught in Units 4.4 to 4.6 and 3.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **10 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 D I · STATE THE OUTPUT · 10 MARKS, PART ONE

The answer: Perfect: 2, Abundant: 5, Deficient: 23

The reasoning, which is worth writing out because "no need to show workings" does not mean "do not check":

Class	Count	Which
Perfect	2	6 ($1+2+3 = 6$) and 28 ($1+2+4+7+14 = 28$)
Abundant	5	12 (16), 18 (21), 20 (22), 24 (36), 30 (42)
Deficient	23	everything else, including every prime, and 1

Two checks that catch mistakes. $2 + 5 + 23 = 30$, so every number is classified exactly once. And 1 is deficient: its proper divisors sum to 0, because 1 has no proper divisor, and the loop for (`div = 1; div < num; div++`) never runs for `num = 1`. Forget that and you get 22, not 23.

CODE · CONFIRMED, WITH THE CLASSIFICATION PRINTED · VERIFIED ON THE JVM

```
public class NumberClassifier {    public static void main(String[] args) {        int p
```

OutputPerfect: 2, Abundant: 5, Deficient: 23

check: 30 numbers classified

CODE · Q2 D II · FOR LOOPS CONVERTED TO WHILE LOOPS · VERIFIED ON THE JVM

```
public class NumberClassifierWhile {    public static void main(String[] args) {
```

OutputPerfect: 2, Abundant: 5, Deficient: 23

Identical output, which is the whole test. Note `int sum = 0;` stays **inside** the outer loop: it must be reset for every number, and moving it out is the classic way to wreck this conversion. The recipe only moves the loop's **own** three parts; everything else stays exactly where it was, which is what "retain other things" means.

Question Three: 2025/2026, Question Six, 12 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Six

THE COURSE TEXT, UNEDITED

(12 marks)A. Design and implement a program, using `JOptionPane` only for input and output, that classifies a single user-supplied positive integer `N` according to its digit pattern (Palindrome).

Definition: A number is a palindrome if it reads the same forwards and backwards (i.e. it is equal to its own digit-reversed value). For example, 4884 and 7 are palindromes; 4821 is not.

Constraints: The entire task must be implemented inside `main()` only, using loops and nested/chained if-else and/or switch logic. No arrays and no user-defined helper methods are permitted; everything must be inline.

i. Using a loop built only on the `%` and `/` operators (no converting `N` to a `String` or array), compute, in the same loop:

- the digit sum of `N`, and
- the value of `N` with its digits reversed (e.g. for `N = 4821`, the reversed value is 1284).

ii. Determine whether `N` is a palindrome by comparing `N` directly to the reversed value computed in step 1.

iii. Inline in `main()` (not a separate method), determine whether the digit sum from step 1 is itself a prime number. Use a loop that tests divisibility only up to the square root of the digit sum.

iv. Using nested/chained conditional logic, classify `N` into exactly one of the following categories, checked strictly in this priority order:

- "PALINDROME-PRIME-SUM": if `N` is a palindrome AND its digit sum is prime
- "PALINDROME": if `N` is a palindrome but its digit sum is not prime
- "HARSHAD": if `N` is not a palindrome but `N` is exactly divisible by its own digit sum
- "STANDARD": in every other case

v. Display `N`, its digit sum, its reversed value, and its final classification in a single `JOptionPane` message dialog.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A definition, worked examples, and a hard set of constraints.

Asked for One inline program. **No helper methods and no arrays**, which is unusual and deliberate.

The marks 12 marks.

Watch Part i says compute both values **in the same loop**, so one pass produces the digit sum and the reversal together. Part iv says "**checked strictly in this priority order**": an if-else chain in that exact order, because a number can satisfy two categories at once.

Taught in Units 3.6, 4.2 to 4.5 and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **12 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 A · CLASSIFY A NUMBER BY ITS DIGIT PATTERN · 12 MARKS

1. in **one loop**, built only on % and / (no converting N to a String or array), compute **both** the digit sum of N and N with its digits **reversed**;
2. N is a palindrome if it equals its reversed value;
3. inline, decide whether the **digit sum** is prime, testing divisibility only up to its **square root**;
4. classify, strictly in this priority order: **PALINDROME-PRIME-SUM** (palindrome and sum prime), **PALINDROME** (palindrome, sum not prime), **HARSHAD** (not a palindrome, but N divides exactly by its digit sum), **STANDARD** otherwise;
5. show N, its digit sum, its reversed value and its classification in a **single** dialog.

MUST MEMORISE · THE DIGIT LOOP, AND THE COPY THAT SAVES IT

Peeling digits off a number is three lines you should be able to write without thinking:

```
int digit = temp % 10;  takes the last digit
reversed = reversed * 10 + digit;  pushes it onto the right of the answer
temp = temp / 10;  drops the last digit
```

Loop while (temp > 0). Both answers fall out of one pass.

The line the marks hang on is `int temp = n;`. The loop destroys whatever it walks: temp ends at 0. Walk n itself and by step 2 there is nothing left to compare the reversed value against, and nothing left to divide in step 4. **Copy it, consume the copy.**

`reversed * 10 + digit` is worth a moment. Take 4821: the loop pulls 1, then 2, then 8, then 4. Start at 0, and each step shifts what you have one place left and drops the new digit into the units: 1, then 12, then 128, then 1284. The digits come off backwards, which is exactly why pushing them on in the order they arrive reverses the number.

MEMORY HOOK: % takes the last digit, / removes it. Every digit problem in this course is those two lines and a loop.

TEACH · THE SQUARE ROOT TEST, WITHOUT MATH.SQRT

Part 3 says test divisibility only up to the square root of the digit sum. The clean way needs no square root at all:

```
boolean sumIsPrime = (digitSum >= 2);for (int f = 2; f * f <= digitSum; f++) {    if (di
```

`f * f <= digitSum` is "f is at most the square root of digitSum", squared on both sides. It stays in whole numbers, so there is no `Math.sqrt`, no `double`, and no floating-point comparison to get wrong at the boundary (Unit 3.6).

Why stopping at the square root is correct, not just faster. If `digitSum = a * b`, then a and b cannot both be above the square root, or their product would exceed digitSum. So every factor pairs with one at or below the square root. Find none there and there are none at all.

And the initialiser is the whole edge case. `digitSum >= 2` is what makes **1 not prime** and **0 not prime**. The loop cannot help you: for `digitSum = 1` it never runs, so whatever you started with survives. Start at true and the program declares 1 prime, and then `N = 1` (a palindrome with digit sum 1) is misclassified. See the table below, last row.

CODE · QUESTION SIX, PART A · VERIFIED ON THE JVM

```
import java.util.Scanner; public class PalindromeClassifier {     public static void main(
```

OutputN: 4884

Digit sum: 24

Reversed: 4884

Classification: PALINDROME

4884 is the paper's own example. It is a palindrome, and $24 = 2 \times 12$ is not prime, so it is PALINDROME and not PALINDROME-PRIME-SUM. That is the priority order doing its job.

CODE · THE SAME STATEMENTS, RUN OVER EVERY CASE · SO THE TABLE IS REAL

This is **not** the exam answer: the question forbids helper methods and demands a single N. It is the identical block of statements with a loop around it, so that every row below is real output rather than someone's arithmetic.

```
import java.util.Scanner; // The SAME statements as the answer, with a loop around them,
```

Output	N	sum	reversed	classification
4884	24	4884	PALINDROME	
7	7	7	PALINDROME-PRIME-SUM	
4821	15	1284	STANDARD	
21	3	12	HARSHAD	
131	5	131	PALINDROME-PRIME-SUM	
1	1	1	PALINDROME	

Six rows, and every branch of part 4 is exercised. **4884** and **4821** are the paper's own two examples and both come out as the paper says. **7** is its one-digit palindrome. **4821**: the digit sum is 15 and $4821 = 15 \times 321 + 6$, so it is not a Harshad number either, and it falls to STANDARD. **1** is the row worth staring at: a palindrome whose digit sum is 1, and 1 is **not** prime, so PALINDROME. Get the initialiser wrong and this row alone would read PALINDROME-PRIME-SUM.

TRAP · "PRIORITY ORDER" MEANS ELSE-IF, NOT FOUR IFS

Part 4 says **checked strictly in this priority order** and **exactly one** category. Both phrases point at the same structure: one chain, where reaching a test means every test above it already failed.

Write four independent ifs and a palindrome with a prime digit sum matches the first, then falls into the others too, and your single dialog reports whichever assignment landed last. The chain is what makes "exactly one" true, and it is also what makes HARSHAD's rule readable:

HARSHAD never has to say "and is not a palindrome", because it is only ever reached when the palindrome test failed. The order does that work, which is precisely why the question fixes it (Unit 4.2).

Watch `n % digitSum` in the HARSHAD test. It is safe here only because `N` is a **positive** integer, so `digitSum` is at least 1 and never 0. Take that guarantee away and it throws `ArithmeticException` (Unit 3.6): a division by zero hiding in a remainder.

Question Four: 2024/2025, Question One, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question One

THE COURSE TEXT, UNEDITED

[4 marks]C. Dry run this code snippet with the following values: 3,4,5,7.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	A method and four values to feed it.
Asked for	A dry run , the lecturer's own word for tracing by hand and stating what comes out. Four values means four answers.
The marks	4 marks, four values: one each. Show the trace, because a bare number cannot earn method marks if you slip.
Watch	Look at the <i>types</i> before you compute anything. Both operands are <code>int</code> , so this is integer division, and that decides the whole answer.
Taught in	Unit 3.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 C · DRY RUN SUMSERIES WITH 3, 4, 5, 7 · 4 MARKS

The answer is 0. For all four values.

MUST MEMORISE · FOUR VALUES, ONE ANSWER, AND THAT IS THE QUESTION

`j` and `j + 1` are both `int`, so `j / (j + 1)` is **integer division**.

`j` is always **smaller** than `j + 1`. A smaller `int` divided by a larger `int` is **always 0**. Every term is 0. The sum is 0.

1/2 is 0. 2/3 is 0. 3/4 is 0. Not 0.5, 0.67, 0.75: **0**, because integer division throws the remainder away (Unit 3.6). It does not round; it truncates.

Why the examiner gave you four values. Being handed 3, 4, 5 and 7 shouts "four different answers". It is bait. The four values are there so that you either notice the types once and answer all four in one line, or trace fractions by hand for ten minutes and write four confident wrong numbers.

MEMORY HOOK: int / int is an int. Before you trace any division, look at the two types. One double anywhere on top and the answer changes completely: $1.0 / 2$ is 0.5.

CODE · THE PAPER'S OWN METHOD, RUN · VERIFIED ON THE JVM

```
public class SumSeriesTrace {    // exactly as the paper prints it    public static int  
OutputsumSeries(3) = 0  
sumSeries(4) = 0  
sumSeries(5) = 0  
sumSeries(7) = 0  
and the terms themselves:  $1/2 = 0$ ,  $2/3 = 0$ ,  $3/4 = 0$   
with a double on top:     $1.0/2 = 0.5$ 
```

Four inputs, one answer. The last two lines are the whole explanation: every term really is 0, and the single change that would rescue the method is a double on top.

DRY RUN · HOW TO WRITE THIS UP FOR 4 MARKS

Do not just write "0" four times. Show the trace for one value and generalise; that is what the marks are for.

sumSeries(3): the loop runs for $j = 1, 2, 3$

j	$j / (j + 1)$	in real arithmetic	as int division	sum
1	$1 / 2$	0.5	0	0
2	$2 / 3$	0.667	0	0
3	$3 / 4$	0.75	0	0

Then one sentence: **"for any i, every term $j/(j+1)$ has $j < j+1$, so integer division gives 0 every time; therefore sumSeries returns 0 for 3, 4, 5 and 7 alike."** That sentence is the answer to all four values, and it is worth more than four traced tables.

Question Five: 2024/2025, Question Two, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Two

THE COURSE TEXT, UNEDITED

[4 marks]C. Dry run this code. State the output for each iteration and state

the final output

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

- Given A complete class with a loop over an array.
- Asked **Two things:** the value at *each iteration*, and the *final* output. A single number answers for half of it.
- The marks 4 marks. The per-iteration trace is what most of them are for, so draw the table.
- Watch Read the loop bounds: it starts at $i = 1$, not 0, because element 0 is already in x. Starting at 0 is the standard slip here.
- Taught in Units 6.3 and 4.4.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 C · DRY RUN THE SUBTRACTION LOOP · 4 MARKS

The question asks for **the output of each iteration and the final output**, which is a specific instruction: it wants the table, not just the number.

DRY RUN · $x = x - \text{nums}[i]$, STARTING FROM $\text{nums}[0]$

$\text{nums} = \{2, 4, 6, 8\}$, and x starts at $\text{nums}[0] = 2$

i	nums[i]	$x = x - \text{nums}[i]$	x after
-	-	$x = \text{nums}[0]$	2
1	4	$2 - 4$	-2
2	6	$-2 - 6$	-8
3	8	$-8 - 8$	-16
Final output			-16

Two things the marker is watching for. First, **the loop starts at $i = 1$, not 0**, because x was already primed with $\text{nums}[0]$; start at 0 and you subtract 2 from itself and get -18. Second, **the loop prints nothing**: the `println` is outside it, so "the output of each iteration" is a trace of x , and the only thing the program actually prints is -16. Say both.

This is the "prime with the first element, then start the loop at 1" idiom you use for maximum and minimum (Unit 8.4). Here it is used for a running subtraction, which is the same shape with a different

operator.

CODE · THE PAPER'S OWN CODE, RUN · VERIFIED ON THE JVM

```
public class SubtractDown {    public static void main(String[] args) {        int[] num
```

Output-16

Question 6: 2024/2025, Question Four, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Four

THE COURSE TEXT, UNEDITED

[3 marks]A. Describe the following statements: Decision-making, Looping and Branching.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing. Recall.

Asked Three descriptions. "Describe" invites one more sentence than "define", so name the for statement *and* what it does.

The marks 3 marks, one per term.

Watch Branching and decision-making sound identical. Keep them apart: deciding *chooses* a path, branching *jumps* out of the normal flow (break, continue, return).

Taught in Units 4.1 to 4.8.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 A · DESCRIBE DECISION MAKING, LOOPING AND BRANCHING · 3 MARKS

Model answer. These are the three families of **control structure** (Unit 4.1): the statements that decide **which** code runs and **how often**, rather than doing the work themselves.

1. **Decision-making.** Choosing between paths by testing a condition, so that some code runs and some does not: `if`, `if-else`, the `if-else` ladder, and `switch`.
2. **Looping (iteration).** Repeating a block while a condition holds: `for`, `while` and `do-while`. The `do-while` is the one that always runs **at least once**, because it tests at the bottom.
3. **Branching (jumping).** Transferring control out of the normal flow: `break` leaves a loop or a `switch` case, `continue` skips to the next iteration, and `return` leaves the method entirely.

Where the marks are: name the actual keywords. "Looping means repeating" is the idea; "for, while, do-while, and do-while tests at the bottom so it runs at least once" is the answer. Units 4.1 to 4.7.

Question 7: 2024/2025, Question Four, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Four

THE COURSE TEXT, UNEDITED

[3 marks]B. Debug this code snippet, list and correct the errors.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A fragment, not a full class.

Asked for List **and** correct.

The marks 3 marks for the set.

Watch The deadliest fault here compiles perfectly: the **stray semicolon** after `while(count <=5)` makes an empty loop body and hangs the program forever. Say so explicitly.

Taught in Units 4.4, 4.5 and 3.2.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 B · DEBUG THE WHILE LOOP · 3 MARKS

The paper prints that as a bare fragment. To put it through a real compiler it needs a class and a main around it, so that is what is added below, and **nothing else is touched**. Every fault you are about to list is the paper's own:

```
public class WhileDebugBroken {    public static void main(String[] args) {        int c
```

Compiler outputerror: ';' expected

The errors, listed.

1. `int count = 1` **has no semicolon**.
2. `float output = ""`; **is the wrong type**. A float is a number; "" is text. They are not interchangeable and Java will not convert one to the other. Fix: `String output = ""`;
3. `while(count <=5);` **has a stray semicolon**. This is the one worth the mark. That semicolon **is** the loop body: an empty statement. The block below it then runs once, on its own, as an ordinary block. Since `count` never changes inside the empty body, the condition stays true forever and the program **hangs**.
4. `Count` **is not a statement**. Wrong case (Java is case sensitive) and, even spelled right, `count` alone does nothing. Fix: `count++`; which is what stops the loop.

TRAP · THE SEMICOLON THAT EATS THE LOOP BODY

`while (condition);` and `for (...);` are both **legal Java**. The semicolon is an empty statement, and an empty statement is a perfectly good loop body. So the compiler says nothing at all.

What you wrote:

```
while (count <= 5); { ... }
```

What Java read:

```
while (count <= 5) { } then, separately, { ... }
```

The braces you meant as the body became a free-floating block that runs **once**, after the loop finishes. Except the loop never finishes, because nothing inside an empty body can change `count`.

This exact trap appears **twice** on this paper: here, and again in Q6 C as for `(int i = 0; i <= n; i++)`; The examiner is not being subtle about how much they want you to notice it. **Whenever a loop misbehaves, look at the character immediately after the closing bracket**. Unit 4.5.

CODE · Q4 B CORRECTED · VERIFIED ON THE JVM

```
public class WhileDebug {    public static void main(String[] args) {        int count =
```

Output1 2 3 4 5

There is a trailing space after the 5, because the loop appends `count + " "` every time including the last. Harmless, and worth writing down if you are asked for the output exactly.

Question 8: 2024/2025, Question Four, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Four

THE COURSE TEXT, UNEDITED

[4 marks]C. State the output of each iteration and the final sum.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A fragment with an array containing a negative value.

Asked for Per-iteration values **and** the final sum. Two deliverables.

The marks 4 marks, mostly for the trace table.

Watch break leaves the loop **immediately**, so the 2 after the negative is never added. Note also that the program prints only once, at the end: "the output of each iteration" means the running value, not printed lines.

Taught in Units 4.7 and 6.3.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 C · STATE THE OUTPUT OF EACH ITERATION AND THE FINAL SUM · 4 MARKS

The answer: Sum = 7.

DRY RUN · BREAK STOPS THE LOOP DEAD

`inputs = {3, 4, -1, 2}`

i	inputs[i]	negative?	sum += inputs[i]	sum
---	-----------	-----------	------------------	-----

0	3	no	0 + 3	3
1	4	no	3 + 4	7
2	-1	yes, so break	never runs	7
3	2	never examined at all		7

The 2 is the question. break leaves the loop **entirely**, so the 2 at index 3 is never seen and never added. Answer 9 and you have read break as continue, which is the whole point of the question.

break **ends the loop**. continue **ends the iteration**.

Swap them here and the sum is $3 + 4 + 2 = 9$, because continue would skip only the -1 and carry on to the 2.

Note also that this loop prints nothing per iteration: the println is after it. "The output of each iteration" is the trace of sum; the program's only output is Sum = 7. Unit 4.7.

CODE · THE PAPER'S OWN CODE, RUN · VERIFIED ON THE JVM

```
public class BreakSum {    public static void main(String[] args) {        int sum = 0;

        OutputSum = 7
```

Question 9: 2024/2025, Question Six, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Six

THE COURSE TEXT, UNEDITED

[3 marks]A. Write a code snippet to demonstrate the following loop types:

- i. A for loop that prints numbers 1 to 5
- ii. A while loop that prints numbers from 5 to 1
- iii. A do-while loop that prints only the number 10 once.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	Nothing.
Asked for	Three snippets, one per loop type. Snippets, not programs.
The marks	3 marks, one per loop.
Watch	Part ii counts downwards , so the update is --. Part iii is the classic do-while argument: it runs the body before testing, which is exactly why it prints once.
Taught in	Units 4.4 to 4.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 A · SNIPPETS FOR THE THREE LOOP TYPES · 3 MARKS

Part iii is the one being marked. A **do-while** whose condition is false still runs **once**, and "print 10 once" is the cleanest possible demonstration of that. Choose a condition that is false immediately, and the body still prints.

CODE · Q6 A · VERIFIED ON THE JVM

```
public class ThreeLoopTypes {    public static void main(String[] args) {        // i. a
```

```
Output1 2 3 4 5
```

```
5 4 3 2 1
```

```
10
```

In the do-while, k is 10, the body prints it, k becomes 11, and while (k <= 10) is **false**, so it stops. The body ran once **before** the condition was ever looked at, which is the whole difference between do-while and while. Write it as a while with the same condition and nothing prints at all. Unit 4.5.

Question 10: 2024/2025, Question Six, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Six

THE COURSE TEXT, UNEDITED

[4 marks]C. Debug this code snippet. List the errors and state the correct output.

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A listing that does not compile.

Asked for List the errors **and** state the output after correction. Two deliverables again.

The marks 4 marks.

Watch Two faults hide in plain sight: the **stray semicolon** on the for line detaches the body, and `main(String args)` is missing its brackets so it is not a valid entry point. Also `printEven(10.0)` passes a double to an int parameter.

Taught in Units 4.4, 5.1 and 3.3.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 C · DEBUG EVENFINDER · 4 MARKS

Compiler outputerror: ';' expected

The errors, listed.

1. `main(String args)` is **not main**. It must be `String[] args`. Without the brackets it is an ordinary method that the JVM will not recognise as the entry point.
2. `printEven(10.0)` **has no semicolon**.
3. `10.0` is a **double**. The method takes an `int`, and Java will not narrow a double to an `int` on its own, because that loses information (Unit 3.3). Fix: `printEven(10);`
4. `printEven(int)` **has no parameter name**. Fix: `printEven(int n)`, which is also why n "does not exist" three lines later.
5. `for (...);` **has a stray semicolon**. The loop body is empty, so the `if` below is **not in the loop**. It runs once, after the loop, and it cannot even compile: `i` was declared in the `for`, so it is out of scope the moment the `for` ends.

Error 5 is Q4 B's stray semicolon again, on the same paper, in a `for` this time. The scope error is what gives it away here: when the compiler says "cannot find symbol: i" about a variable you can plainly see, the loop that declared it has already closed.

The output after correction: 0 2 4 6 8 10

CODE · Q6 C CORRECTED · VERIFIED ON THE JVM

```
public class EvenFinder {    public static void main(String[] args) {        printEven(1
```

Output0 2 4 6 8 10

0 is even, and $i \leq n$ means 10 is included, so the answer starts at 0 and ends at 10. Both ends are worth checking: this is the rare loop where $\leq$ is correct, because n is a value to reach, not a length.

Question 11: 2024/2025, Question Six, 7.5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Six

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program that includes a method called `hasEqualAdjacentPairs(int n)` which:

- i. Returns true if the number n (a positive integer) contains at least two pairs of equal adjacent digits.
- ii. Returns false otherwise.
- iii. You must:
 - a. Write the method `hasEqualAdjacentPairs(int n)`
 - b. Write the `main()` method to test at least two cases

Example

- 112233 – true (pairs: 11, 22, 33). More than two adjacent pairs.
- 1223 – true (pairs: 22). Only one pair – return false

Reproduced exactly. The second example is self-contradictory as printed: it says "true" and then "return false". The rule in part i is the one to trust. This is discussed in the answer below.

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A named method to write, and two worked examples.

Asked for A method **and** a main that tests at least two cases.

The marks 7.5 marks. The "at least two cases" clause is a real instruction: no test calls, no marks for it.

Watch **The paper contradicts itself in the second example.** Part i is unambiguous, "at least two pairs", so 1223 has one pair and must return false. The word "true" in that bullet is an error in the paper. Answer to the stated rule and say in one line why.

Taught in Units 3.6, 4.4 and 5.4.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 D · HASEQUALADJACENTPAIRS · 7.5 MARKS

112233 → true (pairs: 11, 22, 33). More than two adjacent pairs.

1223 → true (pairs: 22). Only one pair → return false

TRAP · THE PAPER CONTRADICTS ITSELF, AND YOU MUST ANSWER ANYWAY

Read the second example again. It says "**1223 - true**" and then, in the same line, "**Only one pair - return false**". Both cannot be right.

The **specification** says "at least two pairs". 1223 has **one** pair, the 22. One is not at least two.

The correct return is false. The leading "true" is a slip of the pen.

The examiner's own explanation ("Only one pair") agrees with the specification, so the specification and the reasoning outvote the stray word. This book shows you the paper exactly as printed and tells you what is true, and it never quietly corrects a source (compare the six errors in Module 3, which come from this course's own slides).

What to do in the hall. Do not agonise, and do not answer the typo. Implement the specification, and **write one line saying why**: "1223 has only one adjacent pair, so by the stated rule of at least two pairs the method returns false; the 'true' in the example appears to be a typographical error." That sentence costs ten seconds and it protects you whichever way the marking scheme reads, because it proves the answer was a decision and not a mistake.

MEMORY HOOK: when a question contradicts itself, follow the specification and say that you did.

CODE · Q6 D, COMPLETE · VERIFIED ON THE JVM

```
public class AdjacentPairs {    // i/ii. true when n has AT LEAST TWO pairs of equal adj  
  
Output112233 -> true  
1223 -> false  
1122 -> true  
111 -> true  
123 -> false  
7 -> false  
122333 -> true
```

The paper asks for at least two test cases; there are seven, and each earns its place. **112233** and **1223** are the paper's own. **1122** is the boundary: exactly two pairs, so exactly true. **111** asks what a triple counts as. **123** has no pairs and **7** has no adjacent digits at all, so the loop never runs and pairs stays 0.

MUST MEMORISE · COMPARE ADJACENT DIGITS BY REMEMBERING THE LAST ONE

Take the last digit **before** the loop and call it previous. Then each turn: take the new current, compare it with previous, and **make current the new previous**.

That is the whole technique, and it is the same % 10 and / 10 pair as Q6 A on the 2025/26 paper. The only new idea is carrying **one** value forward so that each digit can see its neighbour. You never need the digits in an array; you need exactly one of them remembered.

111 gives two pairs, not one. The digits are compared in overlapping pairs: positions 1-2 and 2-3. The middle 1 belongs to both. If you think a triple should count once, that is a different specification, and this one does not say so: it says pairs of equal adjacent digits, and 111 contains two of them.

MEMORY HOOK: to compare neighbours, remember one. Peel a digit, compare, then let it become the memory for the next.

Part 2: the method behind these, taught from nothing

Learn by Doing: Methods

Topic Four

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question One, 2 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question One

THE COURSE TEXT, UNEDITED

[2 Marks] A. Define method overloading in Java, and state precisely the role that the method signature plays in enabling the compiler to resolve an overloaded call at compile time.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing. Recall.

Asked **Two things joined by "and"**: a definition, then the *mechanism*. "State precisely the role" for is asking how the compiler decides, not what overloading is.

The 2 marks: one for the definition, one for the mechanism. Answering only the first half caps marks you at one.

Watch The word **"at compile time"** is the hinge. Overloading is resolved statically, by signature; overriding is resolved at run time. Say "compile time" out loud in your answer.

Taught in Units 5.6 and 5.2.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **2 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 A · DEFINE METHOD OVERLOADING, AND THE ROLE OF THE SIGNATURE · 2 MARKS

Model answer.

Method overloading is a feature that allows a class to have more than one method with the **same name** but **different parameter lists** (differing in the number, types, or order of parameters).

The **method signature** is a method's **name together with its parameter list**. Its role in resolution is this: at a call such as `add(2, 3)`, the compiler compares the number, types and order of the **arguments written at the call site** against the parameter list of every candidate with that name, and selects the one whose signature matches (widening the argument types if no exact match exists). Because the resolution uses only what appears at the call site, and the **return type is not part of the signature**, the return type plays **no role**: two methods differing only by return type are rejected by the compiler as a duplicate definition. The choice is therefore made entirely at **compile time**, which is why overloading is called compile-time (static) polymorphism.

Where the marks are: one for the definition, one for "signature = name + parameter list" and the fact that the return type is excluded. Saying "the compiler picks the right one" without saying *how* is half an answer. See Units 5.1 and 5.6.

Question Two: 2024/2025, Question One, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question One

THE COURSE TEXT, UNEDITED

[3 marks]A. Define Method signature, Method overloading and Void Method.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	Nothing. This is pure recall, no code to read.
Asked for	Three definitions, and they are three separate terms, not one theme. "Define" wants a statement of what the thing <i>is</i> , not an example of it.
The marks	3 marks, three terms: one mark, one sentence each . An essay on overloading cannot earn the mark for void method.
Watch	Signature and overloading are set together on purpose: the second is defined <i>by</i> the first. Define signature first and overloading writes itself.
Taught in	Units 5.1, 5.2 and 5.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 A · DEFINE METHOD SIGNATURE, OVERLOADING, AND VOID METHOD · 3 MARKS

Model answer. One mark each, and each is one sentence.

1. **Method signature.** A method's **name together with its parameter list** (the number, types and order of its parameters). The return type is **not** part of the signature. It is what the compiler uses to tell one method from another.

2. **Method overloading.** Declaring more than one method with the **same name** but **different parameter lists** in the same class, so the compiler chooses between them at compile time by matching the arguments at the call site against each signature.
3. **Void method.** A method declared **void**, meaning it **returns no value**. It is called as a statement on its own, cannot be used in an expression, and may use a bare **return;** to exit early.

Where the marks are: the examiner asked for signature and overloading in the same breath deliberately, because the second is defined by the first. Say "signature = name + parameter list, return type excluded" and the overloading definition writes itself. Units 5.1, 5.2 and 5.6.

Question Three: 2024/2025, Question One, 3 marks

Question Four: 2024/2025, Question One, 7.5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question One

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program that overloads a method named `area` to compute:

- i. The area of a circle
- ii. The area of a rectangle
- iii. The area of a triangle.

Call these methods and print out their values in the main method.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing. You write the whole program.

Asked Three methods that **share the name `area`** and differ in parameter list, plus a main that
for **calls them and prints.**

The marks 7.5 marks. The last sentence is a separate instruction carrying its own marks: three correct methods with no main is an incomplete answer.

Overloading is resolved by the **parameter list**. Circle takes one argument, rectangle and Watch triangle take two, so rectangle and triangle collide unless you distinguish them. Decide how before you start writing.

Taught in Units 5.6 and 5.7.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 D · OVERLOAD AREA FOR CIRCLE, RECTANGLE AND TRIANGLE · 7.5 MARKS

MUST MEMORISE · THE OBVIOUS ANSWER TO THIS QUESTION DOES NOT COMPILE

Circle needs one number. Rectangle needs two. Triangle needs two.

So rectangle is `area(double, double)`, and triangle (base and height) is **also** `area(double, double)`.

That is the same signature. It will not compile.

Overloading needs different **parameter lists**, and "base, height" is not different from "length, width": the parameter *names* are not part of the signature, only the **types** (Q1 A, which you just answered). Write both and javac says:

```
error: method area(double,double) is already defined in class
```

The resolution: give the triangle a **three-number** formula, so its signature is `area(double, double, double)` and distinct. Heron's formula takes the three sides:

$$s = (a + b + c) / 2 \quad \text{and} \quad \text{area} = \sqrt{s(s-a)(s-b)(s-c)}$$

MEMORY HOOK: overloads differ by types, not by names. If two of your methods take the same types in the same order, you have one method and a compiler error.

This is the single most valuable thing in Question One, and the examiner may not have noticed it. Say it explicitly in your answer: "base and height would collide with length and width, so the triangle is done by Heron's formula on three sides, giving a distinct signature." That sentence proves you understand overloading, which is what the question is for.

CODE · Q1 D, COMPLETE · VERIFIED ON THE JVM

```
public class AreaOverload {    // i. circle: one number, so the signature is area(double
OutputCircle, radius 7        : 153.93804002589985
Rectangle, 5 by 3             : 15.0
Triangle, sides 3 4 5         : 6.0
```

Check the triangle without trusting Heron. 3, 4, 5 is a right triangle, so its area must be half of 3×4 , which is **6**. Heron gave exactly 6.0. When you use a formula you cannot see through, feed it a case whose answer you already know. `Math.PI` and `Math.sqrt` are Unit 3.6.

Part 2: the method behind these, taught from nothing

Learn by Doing: Object-Oriented Programming

Topic Five

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question Two, 2 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Two

THE COURSE TEXT, UNEDITED

[2 marks]A. Define encapsulation as an OOP principle

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	Nothing.
Asked for	One definition, framed as an OOP principle , so name the principle and what it buys you.
The marks	2 marks for one term, so two elements are expected: the bundling of data with the methods that act on it, <i>and</i> the hiding of internal state behind them.
Watch	Do not answer "making fields private". That is the mechanism, not the principle. Mention private fields with public accessors as the <i>how</i> , after the <i>what</i> .
Taught in	Units 7.2 and 7.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **2 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 A · DEFINE ENCAPSULATION · 2 MARKS

Model answer.

Encapsulation is the OOP principle of **bundling data (fields) and the methods that operate on them into a single unit (a class), while restricting direct access to some of its components.**

In Java it is achieved by declaring the fields **private** and exposing controlled access through **public** getter and setter methods. This gives **data hiding** (unauthorised code cannot read or modify the state), **validation** (a setter can reject invalid input, for example preventing a negative balance), and **flexibility** (the internal representation can change without breaking any code that uses the class).

Where the marks are: one for the bundling-plus- restricting definition, one for how it is achieved (private fields, public getters and setters). Add one benefit if you have room. See Unit 6.4.

Question Two: 2025/2026, Question Two, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Two

THE COURSE TEXT, UNEDITED

[3 marks]B. State three differences between Method overloading and Method overriding

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing.

Asked for Three contrasts.

The marks 3 marks, one each.

Watch The strongest three are: same class versus superclass and subclass, different parameter list versus identical signature, and compile time versus run time. That last pair is the one that

separates a good answer from a full one.

Taught
in Units 5.6 and 8.4.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 B · THREE DIFFERENCES: OVERLOADING VS OVERRIDING · 3 MARKS

Model answer. Three differences, one mark each:

1. **Where they occur.** Overloading happens **within a single class**: several methods share a name. Overriding happens **between a superclass and a subclass**: the subclass replaces the inherited method.
2. **The parameter list.** Overloaded methods must have **different** parameter lists. An overriding method must have **exactly the same** name and parameter list as the one it overrides.
3. **When the method is chosen.** Overloading is resolved at **compile time** by the compiler (static polymorphism); overriding is resolved at **run time** by the JVM, from the object's actual type (dynamic polymorphism).

Where the marks are: three separate axes. If you have room, add that overriding **requires inheritance** and overloading does not. See Unit 6.7.

Question Three: 2025/2026, Question Four, 25 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Four

THE COURSE TEXT, UNEDITED

A bank wants a system representing different types of accounts using inheritance and polymorphism.

(9 marks)**Part 1: Superclass BankAccount**

- i. Create a class named BankAccount with private fields to hold the account number, the account holder's name, and the balance. Provide a constructor to initialise all three.
- ii. Create a method that adds a given amount to the balance.

iii. Create a method that deducts a given amount from the balance if there are sufficient funds; otherwise, the transaction should be rejected and a message displayed.

iv. Create a method that displays the account's current details: its account number, holder's name, and balance.

(10 marks)**Part 2: Subclasses**

i. SavingAccount extends BankAccount. Override the withdrawal method so a withdrawal is only allowed if the balance after the withdrawal would still be $\geq$ ₦1,000. Otherwise, reject and display a message.

ii. CurrentAccount extends BankAccount. Override the withdrawal method so that every successful withdrawal automatically adds a ₦50 reward to the balance afterward. (6 marks)

Part 3: Main Method

i. Creates at least one SavingAccount and one CurrentAccount;

ii. Deposits money into each;

iii. Performs at least one successful and one unsuccessful/reward-triggering withdrawal on each;

iv. Displays each account's details before and after the transactions;

v. Demonstrates polymorphism by storing each object in a BankAccount reference and calling the withdrawal method through it, confirming that the correct overridden version runs.

The mark allocations are printed as shown: Part 1 is 9, Part 2 is 10, and a further "(6 marks)" is printed against Part 2 item ii. Part 3 carries no printed allocation. Reproduced as it appears.

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A full three-part specification with the class names dictated.

Asked for Three classes and a main that **proves** polymorphism, not merely uses inheritance.

The marks 25 marks, the whole question. Part 3 v is the one that separates answers: it names the technique and tells you exactly how to demonstrate it.

Watch Both subclasses override **the same withdrawal method**, in opposite directions: one restricts it, one rewards it. Write the superclass method so both overrides have something to call, and hold the objects in BankAccount references in main.

Taught in Units 8.1 to 8.6 and 7.2 to 7.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **25 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 · INHERITANCE AND POLYMORPHISM · 25 MARKS

Part 1, BankAccount (9 marks). **Private** fields for account number, holder name and balance; a constructor initialising all three; a method that adds to the balance; a method that deducts if funds suffice and otherwise rejects with a message; a method that displays the details.

Part 2, subclasses (10 marks). SavingAccount overrides the withdrawal so it is allowed only if the balance **after** the withdrawal would still be ≥ 1000 . CurrentAccount overrides it so every **successful** withdrawal adds a 50 reward afterwards.

Part 3, main (6 marks). One of each; deposit into each; at least one successful and one unsuccessful or reward-triggering withdrawal on each; details before and after; and **demonstrate polymorphism by storing each object in a BankAccount reference** and calling the withdrawal through it, confirming the correct overridden version runs.

MUST MEMORISE · THE QUESTION CONTAINS A CONTRADICTION, AND RESOLVING IT IS THE EXAM

Part 1 says the balance must be **private**.

Part 2 says the subclasses must **change the balance**.

A private field is invisible to a subclass. So the subclass cannot touch it.

This is not the examiner being careless. It is the whole of Units 6.4 and 6.6 arriving in one question: **encapsulation says the field is private, and inheritance says the child must still be able to work with it**. The way out is the only way out, and it is the answer being marked: **the subclass goes through the parent's public methods**, using `getBalance()` to read and `super.withdraw(amount)` or `deposit(amount)` to change.

MEMORY HOOK: the child knocks, it does not climb in through the window.

getBalance() to look, super.withdraw() to act.

TRAP · A SUBCLASS CANNOT SEE A PRIVATE FIELD, AND IT WILL SAY SO

The tempting move is to write the arithmetic straight into the subclass. Java stops you at the compiler:

```
class Account {    private double balance = 5000;}class Saving extends Account {    publ
```

Compiler outputerror: balance has private access in Account

Private means the declaring class only, and a subclass is a different class. Inheritance gives Saving the field (every Saving object really does contain a balance) but not **access** to it. The field is there and untouchable, which is exactly what private is for (Unit 6.4).

Would protected fix it? It would compile, and it is the wrong answer **here**, because part 1 said private. Marks are for doing what the question said. Beyond the exam, going through the parent's methods is better anyway: every rule the parent enforces on a withdrawal keeps being enforced. Make the field protected and each subclass has to remember those rules for itself, which is how a bank ends up with an account that can be overdrawn down one code path and not another. See the access table in Unit 6.6.

TEACH · WHY WITHDRAW RETURNS BOOLEAN

The paper does not say what withdraw returns, and the obvious choice is void. Then read part 2 ii again: a CurrentAccount adds the reward on **every successful withdrawal**. Not on every attempt. So the override has to **know whether the withdrawal succeeded**, and a void method tells it nothing.

Making withdraw return boolean makes the override three honest lines:

```
boolean succeeded = super.withdraw(amount);if (succeeded) { deposit(50); }return succeed
```

The alternative, comparing the balance before and after, works but is worse: it re-derives an answer the parent already knew and threw away. **When a method knows something the caller needs, it should return it.** Notice that this decision is forced by a single word in part 2 ii, which is what "read the question" means in practice.

CODE · QUESTION FOUR, COMPLETE · VERIFIED ON THE JVM

```
// ===== Part 1: the superclass =====class BankAccount {    private String accountNumber
```

Output=== iv: details BEFORE the transactions ===

Account SA-1001 | Ada Bello | balance 5000.0

Account CA-2002 | Femi Okonkwo | balance 3000.0

=== ii: deposit into each ===

Deposit 2000.0 -> SA-1001 balance is now 7000.0

Deposit 1000.0 -> CA-2002 balance is now 4000.0

=== iii: a successful and an unsuccessful withdrawal on each ===

Withdraw 4000.0 -> SA-1001 balance is now 3000.0

REJECTED: a saving account must retain 1000. Withdrawing 2500.0 from SA-1001 would leave

Withdraw 1000.0 -> CA-2002 balance is now 3000.0

Reward earned on CA-2002:

Deposit 50.0 -> CA-2002 balance is now 3050.0
 REJECTED: CA-2002 holds only 3050.0, so 999999.0 cannot be withdrawn
 === iv: details AFTER the transactions ===
 Account SA-1001 | Ada Bello | balance 3000.0
 Account CA-2002 | Femi Okonkwo | balance 3050.0
 === v: polymorphism, same reference type, same call, same amount ===
 REJECTED: a saving account must retain 1000. Withdrawing 600.0 from SA-3003 would leave
 Account SA-3003 | Ngozi Eze | balance 1500.0
 Withdraw 600.0 -> CA-4004 balance is now 900.0
 Reward earned on CA-4004:
 Deposit 50.0 -> CA-4004 balance is now 950.0
 Account CA-4004 | Tunde Ade | balance 950.0

All four classes live in one file because only one may be public and that one must match the file name: BankSystem.java. In the hall, write them as four classes exactly as they are here, or as four separate files. Both are correct.

DRY RUN · EVERY BALANCE, AND WHY EACH RULE FIRED

Call	balance before	which rule decided	allowed?	balance after
SA-1001, a SavingAccount, opens at 5000				
deposit(2000)	5000.0	no rule: deposits always land	yes	7000.0
withdraw(4000)	7000.0	7000 - 4000 = 3000, and 3000 >= 1000	yes	3000.0
withdraw(2500)	3000.0	3000 - 2500 = 500, and 500 < 1000	no	3000.0
CA-2002, a CurrentAccount, opens at 3000				
deposit(1000)	3000.0	no rule	yes	4000.0
withdraw(1000)	4000.0	funds suffice, so the reward fires	yes	3000.0, then +50 =
withdraw(999999)	3050.0	funds do not suffice, so no reward	no	3050.0

The last row is the one to study. The reward sits inside if (succeeded), so a rejected withdrawal earns nothing. Had withdraw returned void, that if could not have been written, and CA-2002 would have been paid 50 for a transaction that never happened.

MUST MEMORISE · HOW TO PROVE POLYMORPHISM IN SIX LINES

Part 3 v asks you to **confirm the correct overridden version runs**. A proof needs a control: change **one** thing and show the behaviour changes.

So: same reference type, same method, same amount, same starting balance. Only the object differs.

```
BankAccount acc = new SavingAccount("SA-3003", "Ngozi Eze", 1500.0); acc.withdraw(600.0);
```

Both variables are declared BankAccount. Both calls are acc.withdraw(600.0), character for character. Both accounts hold 1500.0. One is refused and one is paid a bonus.

That is the proof, and here is the sentence that earns the mark: the compiler only knew acc was a BankAccount, so the choice of which withdraw to run could not have been made at

compile time. It was made at **run time**, from the object's **actual** class. That is dynamic method dispatch, and it is what "polymorphism" names (Unit 6.7).

MEMORY HOOK: the reference decides what you may call. The object decides what actually runs.

Compare this with the TRAP at the end of Unit 6.6, where the same setup silently ran the **parent's** method because the override was really an overload. Same two lines of code, opposite outcomes, and the difference was one parameter type. That is why `@Override` sits on every override here.

Question Four: 2024/2025, Question Four, 7.5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Four

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program to define a class called `TemperatureCheck` with the following specifications:

- i. Data field (state): `double temperature`.
- ii. A constructor method to set the temperature
- iii. A method called `evaluateTemperature()` that uses if-else statements to output:
 - a. "Too Cold" if `temperature < 15`
 - b. "Normal" if the temperature is between 15 and 30 (inclusive)
 - c. "Too Hot" if `temperature > 30`.
- iv. In your main method, prompt the user to enter a temperature value, create an object of the class, and display the evaluation result. Use `JOptionPane` to accept inputs.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A full specification: field, constructor, method, main.

Asked A class built to spec. The names are **given to you**, so use them exactly:

for TemperatureCheck, evaluateTemperature().

The 7.5 marks across four parts. Part iv alone carries prompting, object creation, display, and marks JOptionPane.

Watch "between 15 and 30 (inclusive)" means `>= 15 && <= 30`, and the boundary values 15 and 30 are exactly where a marker checks. The paper also insists on **if-else**, so a switch does not answer it.

Taught in Units 7.1 to 7.5, 4.2, and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 D · THE TEMPERATURECHECK CLASS · 7.5 MARKS

MUST MEMORISE · "BETWEEN 15 AND 30 INCLUSIVE" NEEDS NO UPPER TEST

```
if (t < 15) "Too Cold";  
else if (t <= 30) "Normal";  
else "Too Hot";
```

The `else if` only runs when `t < 15` was **false**, which already means `t >= 15`. So writing `else if (t >= 15 && t <= 30)` is not wrong, it is **redundant**: the ladder has already established the lower bound. Let the structure carry what it has proved (Unit 4.2).

The two boundaries are the marks. "Inclusive" means 15 and 30 are both Normal. Get `<` and `<=` the wrong way round and the program is right for 99% of inputs and wrong on exactly the two values the examiner will test.

MEMORY HOOK: test the fences, not the field. When a spec names a boundary, that boundary is the test case.

CODE · Q4 D, COMPLETE · VERIFIED ON THE JVM

```
import java.util.Scanner; // i. the class, with its one data field  
class TemperatureCheck  
  
Output5.0 -> Too Cold  
15.0 -> Normal  
22.0 -> Normal  
30.0 -> Normal
```

31.0 -> Too Hot

40.0 -> Too Hot

Read rows 2 and 4: 15.0 and 30.0 are both Normal, which is exactly what "inclusive" demands, and 31.0 is the first Too Hot. The values were chosen to sit on the fences. In the hall this reads `double t = Double.parseDouble(JOptionPane.showInputDialog("Enter a temperature:"))`; and the result goes out through `showMessageDialog` (F.11); the loop here exists only so one run can show you all six cases.

The field is `private` and set only by the constructor, which is Unit 6.4's encapsulation arriving without being asked for. The question says "data field (state)"; making it `private` costs one word and is never wrong.

Question Five: 2024/2025, Question Five, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Five

THE COURSE TEXT, UNEDITED

[3 marks]A. Describe the following statements: Inheritance, Polymorphism, and Method Overriding.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing. Recall.

Asked for Three descriptions.

The marks 3 marks, one each.

Watch These three are a chain, so describe them as one: inheritance gives you the subclass, overriding replaces a method in it, polymorphism is what lets the replacement run through a superclass reference. Do not define overriding as overloading.

Taught in Units 8.1 to 8.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 A · DESCRIBE INHERITANCE, POLYMORPHISM AND METHOD OVERRIDING · 3 MARKS

Model answer.

1. **Inheritance.** One class acquiring the fields and methods of another with extends, so the subclass has everything the superclass has without repeating it, and may add or replace behaviour. It models an **is-a** relationship: a Bicycle is a Vehicle.
2. **Polymorphism.** One reference, many behaviours: a variable of the superclass type may hold an object of any subclass, and a call through it runs the **object's own** version, decided at **run time**. "Many forms".
3. **Method overriding.** A subclass supplying its own version of a method **the superclass already has**, with the **same signature**, which replaces the inherited one for objects of that subclass.

Where the marks are: these three are one idea in three parts, and saying so earns the mark. **Inheritance** makes the family; **overriding** makes the members differ; **polymorphism** is what you get when you then hold any of them in one variable and call the method. Distinguish overriding (same signature, subclass, run time) from overloading (different parameters, same class, compile time), which is Q1 of this same paper. Units 6.6 and 6.7.

Question 6: 2024/2025, Question Five, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Five

THE COURSE TEXT, UNEDITED

[4 marks]C. Identify the errors in this code snippet and state the final output after correction.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A listing that does not compile.

Asked **Two things: the errors, and the output once fixed.** Stopping at the error list leaves
for marks on the table.

The marks	4 marks, split between finding faults and running the corrected version in your head.
Watch	<code>int nums = {10, 20, 30}</code> is missing its brackets, and <code>nums[3]</code> is out of bounds on a 3-element array. Once corrected, decide what "middle value" should sensibly be before you state an output.
Taught in	Units 6.2 to 6.5.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 C · IDENTIFY THE ERRORS AND STATE THE OUTPUT AFTER CORRECTION · 4 MARKS

Compiler outputerror: not a statement

The errors, listed.

- `int nums = {10, 20, 30};` **is not an array**. A list in braces needs an array type. Fix: `int[] nums = {10, 20, 30};`
- `nums[3]` **does not exist**. Three elements means indexes 0, 1 and 2. And the label says **"Middle value"**, so the intent is `nums[1]`, which is 20. Say both halves: the index is out of bounds **and** the middle of three is index 1.
- `i <= nums.length` **is off by one** and would throw at `nums[3]`. Fix: `i < nums.length`.
- `i` **never increments**. Fix: `i++`.
- `"Sum is: " + );` **has nothing to add**. Fix: `"Sum is: " + sum`.

The output after correction:

OutputMiddle value is: 20
Sum is: 60

CODE · Q5 C CORRECTED · VERIFIED ON THE JVM

```
public class MiddleValue {    public static void main(String[] args) {        int[] nums
OutputMiddle value is: 20
Sum is: 60
```

Errors 3 and 4 are the same two errors as Q2 B on this paper, character for character. **The off-by-one and the missing increment are this examiner's signature.** They are on both papers, in both years. Learn them once.

Question 7: 2024/2025, Question Five, 7.5 marks

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program using inheritance and method overriding as follows:

- i. Create a superclass called `Vehicle` with the following:
 - a. An instance variable `speed` (`int`)
 - b. A constructor that initialises `speed`
 - c. A method `displayInfo()` that prints `"Vehicle speed: <speed> km/h"`
- ii. Create a subclass called `Bicycle` that overrides the `displayInfo()` method to print: `"Bicycle speed: <speed> km/h. This is an eco-friendly ride."`
- iii. In the `main` method:
 - a. Create an object of class `Vehicle` with a speed of 80
 - b. Create an object of class `Bicycle` with a speed of 25
 - c. Call `displayInfo()` on both objects to demonstrate polymorphism.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

- | | |
|-----------|--|
| Given | A full specification, including the exact strings to print. |
| Asked for | Two classes and a main that proves polymorphism. |
| The marks | 7.5 marks. The printed strings are quoted in the paper, so reproduce them exactly, km/h and full stop included. |
| Watch | <speed> is a placeholder for the value, not text to print. And to "demonstrate polymorphism" the objects should be held in <code>Vehicle</code> references, or you have |

demonstrated nothing.

Taught
in

Units 8.1 to 8.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 D · VEHICLE AND BICYCLE · 7.5 MARKS

CODE · Q5 D, COMPLETE · VERIFIED ON THE JVM

```
// i. the superclassclass Vehicle {    protected int speed;    public Vehicle(int speed)
```

```
OutputVehicle speed: 80 km/h
```

```
Bicycle speed: 25 km/h. This is an eco-friendly ride.
```

```
-- through a Vehicle reference --
```

```
Vehicle speed: 80 km/h
```

```
Bicycle speed: 25 km/h. This is an eco-friendly ride.
```

TEACH · THE LAST FOUR WORDS OF THE QUESTION CHANGE THE ANSWER

The question says: call `displayInfo()` on both objects **to demonstrate polymorphism**. Read literally, parts a to c are satisfied by:

```
Vehicle car = new Vehicle(80);Bicycle bike = new Bicycle(25);car.displayInfo();bike.displayInfo();
```

That prints the two required lines, and it demonstrates **nothing**. Each variable's declared type already matches its object exactly, so the compiler could have picked both methods without any run-time decision. Overriding happened, but polymorphism was never tested.

Polymorphism needs the reference to be the SUPERCLASS type, so that the declared type and the actual object disagree and only the run time can settle it:

```
Vehicle[] fleet = {new Vehicle(80), new Bicycle(25)};
for (Vehicle v : fleet) v.displayInfo();
```

Now every element is declared `Vehicle`, the loop body is one identical call, and it still prints two different lines. **That** is the demonstration, and it is what the last four words are asking for. The listing above does both: the literal answer first, then the proof.

Same idea as Q4 part 3 v on the 2025/26 paper, where the reference is explicitly required to be a `BankAccount`. Two papers, two years, one point: **the reference decides what you may call, the object decides what runs**.

Part 2: the method behind these, taught from nothing

Learn by Doing: Arrays

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question One, 5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question One

THE COURSE TEXT, UNEDITED

[5 Marks]C. This code is supposed to: Loop through the array, and for each starting index `i`, find the first number (from `i` onward) that's a multiple of 4. Add (that number - 1) to sum, count it as a match, stop after 3 matches, then display the sum and match count. Find the errors if the correct output should be:

Sum: 141.0 Matches: 3

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A specification in words, a listing, and **the correct output**.

Asked for The errors. The output is handed to you precisely so you can work backwards from it.

The marks 5 marks, and there are more than five faults, so hunt systematically rather than stopping at the first that looks right.

Watch **Read the given output first.** "Sum: 141.0" has a decimal point, but the listing declares `int sum`: that single character tells you the types are wrong before you read a line of logic. Line 4 stores doubles in an `int[]`, and `i <= arr.length` runs off the end.

Taught in Units 3.3, 6.3, 4.4 and 4.7.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 C · FIND THE ERRORS · 5 MARKS

Read the output first. "141.0" tells you the sum is a double before you have read a line of code. That single character finds two of the five errors.

CODE · THE PAPER'S SNIPPET, WITH EVERY ERROR MARKED · AS PRINTED

```
import javax.swing.JOptionPane;public class Snippet {public static void main(String[] ar
```

WORKED EXAMPLE · THE FIVE ERRORS, AND WHY EACH IS ONE

- Line 4: `int[]` cannot hold 24.0.** Those are double literals. This is a **compile** error: "incompatible types: possible lossy conversion from double to int". Fix: `double[] arr`. The output confirms it: 141.0 is a double.
- Line 5: `int sum` cannot total doubles,** and could never print "141.0". Fix: `double sum = 0;` Unit 3.3: the type decides what you may do and how it prints.
- Line 8: `i <= arr.length` is off by one.** Indexes run 0 to length-1 (F.10). Fix: `i < arr.length`. Here it would not actually throw, because the inner loop never runs when `i = 5`, but it is still wrong and it is one of the five marks.
- Lines 10 to 12: the continue is inverted.** The spec says find numbers that **are** multiples of 4. This skips them and processes everything else: precisely backwards. Fix: `if (arr[j] % 4 != 0) continue;` Unit 4.7: read every continue as "unless". This is the error worth the most attention, because it is the one that is pure logic.
- Line 13: `arr[j] - j` subtracts the index.** The spec says add **(that number - 1)**. Fix: `sum += arr[j] - 1;` A subtle one: with `j` it would give the right answer for `j = 1` only, so a spot check could miss it.

CODE · CORRECTED, AND IT REPRODUCES THE PAPER'S ANSWER · VERIFIED ON THE JVM

```
// JOptionPane replaced by println so the output can be printed here;// in the exam, kee
```

OutputSum: 141.0

Matches: 3

That is the proof. The paper printed Sum: 141.0 and Matches: 3, and the corrected program produces exactly that. If your five fixes do not reproduce the stated output, one of them is wrong, and the paper has handed you the means to check your own answer.

DRY RUN · WHY THE ANSWER IS 141

`arr = {24.0, 40.0, 66.0, 80.0, 98.0}`

i	first multiple of 4 from i onward	add (that - 1)	matches	sum
0	<code>arr[0] = 24</code>	<code>24 - 1 = 23</code>	1	23.0
1	<code>arr[1] = 40</code>	<code>40 - 1 = 39</code>	2	62.0
2	66 is not, <code>arr[3] = 80</code> is	<code>80 - 1 = 79</code>	3	141.0
-	<code>matches == 3, so stop = true</code>			141.0

Note $i = 2$ skipped 66 and found 80, which is exactly what the inner loop is for: "the first multiple of 4 **from i onward**". And the outer loop never reached $i = 3$, because **stop** ended it.

Question Two: 2025/2026, Question Two, 5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question Two

THE COURSE TEXT, UNEDITED

[5 marks]C. This code is supposed to loop through an array of prices, skip anything under 20, apply a 10% discount to prices 20 and above, add the discounted price to total, count it, and stop once 4 items have been discounted. Then it displays the total and count. Find the errors if the correct output should be:

Total: 202.725 Count: 4

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A specification, a listing, and the correct output.

Asked
for The errors, worked backwards from the stated output.

The
marks 5 marks.

Same trick as Q1 C, and the same tell: "Total: 202.725" cannot come out of an `int total`.

Watch Then check every comparison against the words: the spec says skip *under* 20 but line 9 skips *over* 20, and it says stop at **4** while line 15 tests for 5.

Taught
in Units 3.3, 6.3, 4.4 and 4.7.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 C · FIND THE ERRORS · 5 MARKS

The same five errors as Q1, in the same order. This is the payoff of the template.

WORKED EXAMPLE · THE FIVE ERRORS

- Line 4:** `int[] prices = {15.5, ...}`. Decimal literals in an int array: a compile error. Fix: `double[] prices`. The output "202.725" says double before you read anything else.
- Line 5:** `int total = 0`; cannot hold 202.725. Fix: `double total = 0`;
- Line 8:** `i <= prices.length` is off by one. Fix: `i < prices.length`. Unlike Q1, this one **would** throw `ArrayIndexOutOfBoundsException` at `i = 6`, because line 9 reads `prices[i]` straight away. Same error, worse consequence.
- Line 9:** `if (prices[i] > 20) continue`; **is inverted**. The spec says skip anything **under** 20. This skips everything **over** 20, i.e. exactly the items to be discounted. Fix: `if (prices[i] < 20) continue`;
- Line 15:** `if (count == 5)`. The spec says stop once **4** have been discounted. Fix: `if (count == 4)`. Only four of the six prices are 20 or above anyway, so with `== 5` the flag never fires. The count would still be 4, and the total would still be right, which makes this the easiest error to miss: it does not change the answer, it just means the code does not do what it was asked.

CODE · CORRECTED, REPRODUCING THE PAPER'S ANSWER · VERIFIED ON THE JVM

```
public class Q2C {    public static void main(String[] args) {        double[] prices =  
OutputTotal: 202.725  
Count: 4
```

Matches the paper exactly.

DRY RUN · WHY THE ANSWER IS 202.725

`prices = {15.5, 42.0, 8.75, 60.0, 33.25, 90.0}`, 10% off each

i	price	under 20?	discounted (90%)	count	total
0	15.5	yes: skip	-	0	0
1	42.0	no	37.8	1	37.8
2	8.75	yes: skip	-	1	37.8
3	60.0	no	54.0	2	91.8
4	33.25	no	29.925	3	121.725
5	90.0	no	81.0	4	202.725

count reaches 4, done becomes true, and the loop ends having consumed the whole array anyway. Total 202.725, Count 4.

Question Three: 2025/2026, Question Three, 12 marks

THE COURSE TEXT, UNEDITED

(12 marks) A. A store records daily sales in an integer array `saleAmounts[]`, in the order the sales happened. A "consecutive duplicate" is any position `i` (`i >= 1`) where `saleAmounts[i] == saleAmounts[i-1]`.

Given the array:

Index	0	1	2	3	4	5	6	7	8	9
Amount	20	20	35	40	40	40	15	60	60	25

Write a program that:

- i. Create a method that makes one pass through the array and returns the count of indices `i >= 1` where `saleAmounts[i] == saleAmounts[i-1]`.
- ii. Create a method that returns a single `String` listing, in original order and comma-separated, every sale amount that equals its predecessor or its successor in the array. Build this string using plain `String` concatenation only (no `StringBuilder`, no array of booleans returned). Index 0 checks only its successor; the last index checks only its predecessor.
- iii. Using `JOptionPane`, display one dialog showing (a) the total consecutive-duplicate count from part 1, and (b) the flagged-amounts string from part 2.

[Hint: To aid your flow trace your algorithm by hand first: it must not flag 35, 15, or 25.]

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A ten-element array, a precise definition, and a hint that doubles as a test case.

Asked Two methods and a dialog. Parts i and ii count **different things**: i counts positions, ii lists for amounts.

The marks 12 marks. Part ii carries most of them and has the most constraints attached.

Watch Part ii bans `StringBuilder` and forbids returning a boolean array, and it spells out the two edge cases (first and last index). The hint tells you the answer must exclude 35, 15 and 25, so you can check yourself before you hand in.

Taught in Units 6.3 to 6.6, 5.4 and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **12 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 A · CONSECUTIVE DUPLICATES IN A SALES LOG · 12 MARKS

Index	0	1	2	3	4	5	6	7	8	9
Amount	20	20	35	40	40	40	15	60	60	25

Write a program that (i) counts, in **one pass**, the indices $i \geq 1$ where the amount equals the one before it; (ii) returns a single String listing, in order and comma separated, every amount that equals its **predecessor or its successor**, built with **plain String concatenation only**; (iii) displays both in **one** JOptionPane dialog. The paper adds a hint: *it must not flag 35, 15, or 25*.

MUST MEMORISE · PARTS I AND II COUNT DIFFERENT THINGS

Part i counts positions. Part ii lists values. They are not the same question asked twice, and they do not give the same number.

Part i asks how many **boundaries** have a twin across them. Part ii asks which **elements** are touching a twin at all.

Look at the three 40s. They contain **two** boundaries (index 3 to 4, and 4 to 5) but **three** elements. So part i counts 2 there and part ii lists 3. Read the two parts as one question and you will answer one of them wrong.

MEMORY HOOK: fences and posts. Part i counts the fences between posts, part ii counts the posts. A run of n twins has n posts and $n-1$ fences, always.

DRY RUN · BOTH ANSWERS, BY HAND, BEFORE WRITING ANY CODE

saleAmounts = {20, 20, 35, 40, 40, 40, 15, 60, 60, 25}

i	amount	same as previous?	same as next?	part i counts?	part ii flags?
0	20	no predecessor	yes (20)	-	yes
1	20	yes (20)	no (35)	yes	yes
2	35	no (20)	no (40)	no	no
3	40	no (35)	yes (40)	no	yes
4	40	yes (40)	yes (40)	yes	yes

5	40	yes (40)	no (15)	yes	yes
6	15	no (40)	no (60)	no	no
7	60	no (15)	yes (60)	no	yes
8	60	yes (60)	no (25)	yes	yes
9	25	no (60)	no successor	no	no
Answers				4	20, 20, 40, 40, 40, 60, 60

Check it against the hint. The paper told you 35, 15 and 25 must not be flagged, and they are the only three that are not. The examiner has handed you a way to test your own trace before you write a line of code, which is exactly what the hint is for.

And check the two answers against each other. The runs are 20-20 (2 long), 40-40-40 (3 long) and 60-60 (2 long). Part ii flags $2 + 3 + 2 = 7$ values. Part i counts $1 + 2 + 1 = 4$ positions. They differ by exactly **3**, the number of runs, because each run loses one to the fences-and-posts rule. If your two answers do not differ by the number of runs, one of them is wrong.

CODE · QUESTION THREE, PART A · VERIFIED ON THE JVM

```
public class SalesDuplicates {    // Part i: ONE pass. Count every index i >= 1 whose arr
OutputConsecutive duplicate count: 4
Flagged amounts: 20, 20, 40, 40, 40, 60, 60
```

Exactly the hand trace, including the three amounts the paper told us not to flag. The dialog is a `println` here so its text can be printed in this book; in the hall, write `JOptionPane.showMessageDialog(null, message);` and import `javax.swing.JOptionPane` (F.11).

TRAP · THREE WAYS TO LOSE MARKS ON PART A

1. Starting part i's loop at 0. The question says $i \geq 1$, and for a reason: `sales[0 - 1]` is `sales[-1]`, which throws `ArrayIndexOutOfBoundsException` (Unit 8.1). Index 0 has no predecessor, so the loop starts at 1. This is where "one pass" is easiest to get right and easiest to throw away.

2. Forgetting the boundary guards in part ii. Index 0 has no predecessor and the last index has no successor. The guards $(i > 0)$ and $(i < \text{sales.length} - 1)$ are what stop the reads from running off either end. They work because `&&` is **short-circuit** (Unit 3.6): when $i > 0$ is false, Java never evaluates `sales[i - 1]`, so it never throws. Swap the two sides of the `&&` and the program crashes on the first element.

3. The leading comma. Concatenating `", " + sales[i]` every time gives `, 20, 20, 40...` with a comma stuck on the front. The test `if (result.equals(""))` writes the first value bare and every later value with its separator in front. Compare Strings with `.equals()`, never with `==` (Unit 7.3).

The question bans `StringBuilder` deliberately: it wants to see that you can handle the separator yourself. Use `StringBuilder` anyway and you have answered a question the paper did not ask.

Question Four: 2025/2026, The cold-storage log, 13 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, The cold-storage log

THE COURSE TEXT, UNEDITED

(13 marks)B. A cold-storage warehouse records its internal temperature once every hour, for 24 hours. Write a program, using JOptionPane only for input/output, that checks the day's temperature log for sustained breaches of a safe limit. A reading is a breach if it is above 8°C.

i. Using a loop, prompt the user 24 times (one dialog per hour) for that hour's temperature. Each input must be validated: it must parse as a number between -20 and 50 inclusive. If invalid, re-prompt for the same hour (using a nested loop/control statement) until a valid value is entered. Store all 24 valid readings in a 1-D array.

ii. Scan the 24-hour array once to find every maximal run of consecutive breach-hours (temperature > 8°C). Using only loops and conditionals (no extra arrays), determine:

- the total number of qualifying streaks, where a qualifying streak is any consecutive run of 3 or more breach-hours;
- the length (in hours) of the single longest breach streak anywhere in the log.

iii. Using nested if-else logic, classify the day as exactly one of:

- "CRITICAL": longest streak ≥ 6 hours
- "WARNING": longest streak is 3–5 hours (inclusive)
- "SAFE": no streak of 3 or more consecutive breach-hours occurred

iv. Show one JOptionPane dialog stating: the total number of qualifying streaks, the longest streak length, and the final classification.

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A 24-hour log, a breach threshold, and three classifications.

Asked for Validated input, a single scan, a classification, and one dialog.

The marks 13 marks.

Watch Two constraints do the real work: "**scan once**" and "**no extra arrays**", which together forbid the obvious approach of collecting streak lengths into a list. Track the current run and the best run as you go. Note also that a breach is strictly *above* 8, so exactly 8 is safe.

Taught in Units 4.2 to 4.6, 6.3 and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **13 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 B · SUSTAINED BREACHES OF A SAFE LIMIT · 13 MARKS

1. prompt 24 times, one dialog per hour. Each input must parse as a number **between -20 and 50 inclusive**; if invalid, re-prompt for the **same** hour using a nested loop, until valid. Store all 24 in a 1-D array;
2. scan the array **once**, using only loops and conditionals (**no extra arrays**), to find the **number of qualifying streaks** (runs of **3 or more** consecutive breach-hours) and the **length of the longest** breach streak;
3. using nested if-else, classify: **CRITICAL** if the longest streak is ≥ 6 , **WARNING** if it is 3 to 5, **SAFE** if no streak of 3 or more occurred;
4. show one dialog with the streak count, the longest length, and the classification.

MUST MEMORISE · COUNT THE RUN WHEN IT TURNS 3, AND MEASURE IT AS YOU GO

The whole of part 2 is three variables and one pass:

currentRun: breach hours in a row right now, reset to 0 on any non-breach.

longestStreak: updated **every breach hour**, not at the end of a run.

qualifyingStreaks: incremented when `currentRun == 3`, **exactly**.

Why `== 3` and not `>= 3`. A run passes through 3 exactly once, on its third hour. So `== 3` counts each qualifying run **once**, however long it becomes. Use `>= 3` and a 6-hour run is counted four times, giving 4 streaks where there is one.

MEMORY HOOK: a run is counted at the moment it qualifies, not while it continues. The threshold is a door you pass through once, not a room you stay in.

TRAP · THE STREAK THAT IS STILL RUNNING WHEN THE DAY ENDS

The instinct is to close each run in the `else`, where the non-breach hour is: "the run just ended, so now measure it."

```
if (readings[hour] > 8) {    currentRun++;} else {    if (currentRun > longestStreak) lo
```

It works, until the day ends mid-streak. If hours 18 to 23 are all breaches, the loop finishes with `currentRun = 6` and **never reaches the else**. The longest streak of the day is never measured and never counted. The program reports SAFE on a day the freezer was over temperature for six straight hours.

The fix is not to bolt a copy of the else-block on after the loop. That is the usual patch and it duplicates the logic in two places, so the next edit changes one and forgets the other. Instead, **update on every breach hour:**

Measure `longestStreak` **as the run grows**, and count the run **the instant it turns 3**. Then a run that is still going when the array ends has already been measured and already been counted, and the end of the array needs no special case at all.

This is worth more than one exam question. **A boundary you never treat specially cannot be got wrong at the boundary.**

CODE · QUESTION FIVE, PART B · VERIFIED ON THE JVM

```
import java.util.Scanner;public class ColdStorage {    public static void main(String[]
```

Output hour 0: 99.0 is outside -20 to 50. Asking again.

hour 0: "abc" is not a number. Asking again.

Qualifying streaks (3+ hours): 3

Longest breach streak: 6 hour(s)

Classification: CRITICAL

The run fed hour 0 a 99, then the text "abc", then a valid 5, so both halves of "must parse as a number between -20 and 50" are shown rejecting input and re-prompting **the same hour**.

DRY RUN · THE 24 HOURS, AND WHERE THE THREE STREAKS ARE

breach = above 8. The bar shows breach hours.

hours	readings	breach run	counted?
0 to 3	5, 9, 10, 4	2 in a row	no: under 3
4 to 7	9, 9, 9, 2	3 in a row	yes, streak 1
8 to 13	12, 11, 9, 9, 10, 3	5 in a row	yes, streak 2
14 to 17	1, 2, 3, 4	none	no
18 to 23	9, 9, 9, 9, 9, 9	6 in a row, to the last hour	yes, streak 3
Answers		longest = 6	3 streaks, CRITICAL

The first run of 2 is the control: it proves the counter is not simply counting runs. The last run is the trap above, live: it reaches hour 23 and stops because the day stopped, not because the

temperature did. It is both the longest streak and the one that decides the classification, and the else-branch version would have missed it entirely and printed WARNING.

Question Five: 2025/2026, The parking garage, 13 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, The parking garage

THE COURSE TEXT, UNEDITED

(13 marks)B. A university parking garage needs a program, using JOptionPane only for input/output, to process a batch of vehicles exiting on the same day and calculate the total fees owed. The program must use clear separation of concerns; each task below must be its own method. No collection classes (e.g. ArrayList) are allowed anywhere.

i. Prompt for the number of exiting vehicles, V. Then, using a loop, fill two parallel arrays of length V:

- the number of hours each vehicle was parked (an integer per vehicle). A value ≤ 0 is invalid; use a nested control statement to keep re-prompting until a valid value is entered.
- the permit type for each vehicle (a String per vehicle, either "Staff" or "Visitor").

ii. Create a method that takes in one vehicle's hours parked and permit type, and computes one vehicle's fee using nested if-else logic (the rules below must be nested together, not checked as separate independent conditions):

- Staff: first 2 hours free; each additional hour costs 1.00; capped at a maximum of 10.00/day.
- Visitor: no free period; first 3 hours cost a flat 6.00; each hour beyond 3 costs 2.50; capped at a maximum of 25.00/day.
- Any permit type that is not exactly "Staff" must be treated as "Visitor".

iii. Create a method that takes in the array of hours parked, loops over the array and tallies vehicles into three categories: SHORT-STAY (1-2 hrs), STANDARD (3-6 hrs), EXTENDED (>6 hrs). Return these three counts without using any collection class.

iv. In main(),

- call the method from part (ii) for every vehicle across both parallel arrays to accumulate the batch's total revenue
- call the method from part (iii) once
- display one final JOptionPane showing the total revenue and the three category counts.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Two fee schedules and three stay categories.

Asked for Parallel arrays, one fee method, one tally method, and a main that calls them.

The marks 13 marks.

Watch Part iii asks one method to **return three counts** with no collection class allowed, which is the real puzzle: an `int[]` of three is the intended way out. Note also the caps are on the fee, not the hours, so compute then clamp.

Taught in Units 6.3 to 6.6, 5.4, 4.2 and F.11.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **13 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 B · PARKING FEES FOR A BATCH OF VEHICLES · 13 MARKS

1. prompt for the number of vehicles `V`, then fill two **parallel arrays** of length `V`: hours parked (int per vehicle; a value `<= 0` is invalid, so use a **nested control statement** to re-prompt until

valid) and permit type (String per vehicle, "Staff" or "Visitor");

2. a method taking one vehicle's hours and permit, computing its fee with **nested if-else**: **Staff**, first 2 hours free, each additional hour 1.00, capped at 10.00/day; **Visitor**, no free period, first 3 hours a flat 6.00, each hour beyond 3 costs 2.50, capped at 25.00/day; and **any permit that is not exactly "Staff" is treated as Visitor**;
3. a method taking the hours array, tallying **SHORT-STAY** (1-2), **STANDARD** (3-6) and **EXTENDED** (>6), returning the three counts **without any collection class**;
4. `main()`: call (ii) for every vehicle across both arrays to accumulate revenue, call (iii) once, and display **one** dialog with the total and the three counts.

MUST MEMORISE · THREE COUNTS, ONE RETURN, NO COLLECTIONS

Part 3 asks for **three** numbers back. A Java method returns **one** value. And ArrayList is banned.

So return an `int[]` of length 3.

An array is not a collection class. That distinction is the question: "collection classes (e.g. ArrayList)" means `java.util`'s List/Set/Map family. An array is a built-in Java type, not a class you import, which is why F.10 and Module 8 exist and why the ban costs you nothing.

Document the slots in a comment (`[0]` short, `[1]` standard, `[2]` extended) and use them in that order. The alternative the ban is steering you away from is three separate fields or three methods walking the same array three times.

MEMORY HOOK: one return value, several numbers, no collections: that is an array.

TEACH · PARALLEL ARRAYS, AND THE INVARIANT THAT KEEPS THEM HONEST

Two arrays are **parallel** when index `i` in one and index `i` in the other describe the same thing. `hoursParked[3]` and `permitType[3]` are one car.

The whole idea rests on one invariant: **they are the same length and they are always written together**. The loop in part 1 fills both in the same iteration, which is what keeps them aligned. Fill them in two separate loops, or `continue` past one but not the other, and vehicle 3's hours are paired with vehicle 4's permit. Nothing throws. Every fee is simply wrong.

In Module 6's terms this is what a class is for: a `Vehicle` object holding hours and permit together cannot fall out of step, because there is no "step" to fall out of. The paper asks for parallel arrays, so write parallel arrays; but knowing what they are the poor relation *of* is what Q4 was testing.

CODE · QUESTION SIX, PART B · VERIFIED ON THE JVM

```
import java.util.Scanner; public class ParkingGarage {    // Part ii: one vehicle's fee,
```

```
Output  vehicle 0: 0 is not a valid number of hours. Asking again.
```

```
Vehicle 0: 2h, Staff -> 0.0
```

```
Vehicle 1: 5h, Staff -> 3.0
```

```
Vehicle 2: 15h, Staff -> 10.0
```

```
Vehicle 3: 1h, Visitor -> 6.0
```

```
Vehicle 4: 5h, Visitor -> 11.0
```

```
Vehicle 5: 12h, Guest -> 25.0
```

```
Total revenue: 55.0
```

```
SHORT-STAY (1-2 hrs): 2
```

STANDARD (3-6 hrs): 2

EXTENDED (>6 hrs): 2

DRY RUN · EVERY RULE IN THE FEE TABLE, FIRED ONCE

hours	permit	which rule, and the arithmetic	fee
2	Staff	first 2 hours free, and 2 is not beyond 2	0.00
5	Staff	$(5 - 2) \times 1.00 = 3.00$, under the 10.00 cap	3.00
15	Staff	$(15 - 2) \times 1.00 = 13.00$, over the cap	10.00
1	Visitor	flat 6.00 for the first 3 hours, even for 1	6.00
5	Visitor	$6.00 + (5 - 3) \times 2.50 = 11.00$	11.00
12	Guest	not exactly "Staff", so Visitor: $6.00 + 9 \times 2.50 = 28.50$, over the cap	25.00
Total revenue			55.00

Every rule in the question fires exactly once in that table, which is how the test data was chosen: both free periods, both per-hour rates, **both** caps, and the not-Staff-means-Visitor rule. When you invent your own test data, invent it like this: one row per rule, so a single run proves the lot.

Two rows repay a second look. **1 hour as a Visitor costs 6.00**: "first 3 hours cost a flat 6.00" is a flat fee, not a rate, so an hour and three hours cost the same, and a Visitor pays more than a Staff member who stayed fifteen. That is not a bug, it is the pricing the question specifies. And **Guest at 12 hours** is the only row testing the fallback rule; delete it from your test data and a program that crashes on unknown permits would look perfectly healthy.

MUST MEMORISE · A CAP IS A CEILING YOU APPLY AFTER, NOT A RULE YOU FOLD IN

Compute the fee, **then** clamp it:

```
fee = 6.00 + (hours - 3) * 2.50;  
if (fee > 25.00) { fee = 25.00; }
```

The temptation is to work out the hour at which the cap bites (for Visitors, 10.6 hours) and branch on **that**. It gives the same answers and it is a worse answer: you have hidden a magic number that no line of the question contains, and changing the rate to 3.00 silently breaks it, because the crossover moves and your constant does not.

This is the same shape as Q5 A's price floor, which is a cap from below. Both read as one line placed after the sum, and both then say plainly what the question said: **capped at a maximum of 25.00/day**.

MEMORY HOOK: compute, then clamp. Never solve for where the clamp starts.

Question 6: 2024/2025, Question Two, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Two

[3 marks] A. Define the following: Dynamic array initialisation, the length property of arrays, and the out-of-bounds exception in arrays

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	Nothing. Recall.
Asked for	Three definitions again, one per term.
The marks	3 marks, one per term, one sentence each.
Watch	length is a property (a field), not a method , so it takes no brackets. Saying "length()" in a definition question throws the mark away.
Taught in	Units 6.1 to 6.4.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 A · DEFINE DYNAMIC ARRAY INITIALISATION, LENGTH, AND THE OUT OF BOUNDS EXCEPTION · 3 MARKS

Model answer.

- 1. Dynamic array initialisation.** Creating the array with new and a size decided **while the program runs**, rather than from a literal list written in the source: `int[] a = new int[n];` where `n` is only known at run time. Every element starts at its default (0 for `int`, `null` for objects), and the values are filled in afterwards. Contrast static initialisation, `int[] a = {2, 4, 6, 8};`, where the size and the values are both fixed in the source.
- 2. The length property.** `a.length` gives the number of elements. It is a **field, not a method**, so there are no brackets: writing `a.length()` is a compile error. It is **final**: an array's length is fixed the moment it is created and can never change. A `String`'s length IS a method, `s.length()`. The examiner knows that catches people; Unit 7.2.
- 3. The out-of-bounds exception.** `ArrayIndexOutOfBoundsException`, thrown at **run time** when you use an index below 0 or at or above `length`. Valid indexes are **0 to length - 1**, so

`a[a.length]` always throws. It is **unchecked**, so the compiler will not warn you about it.

Where the marks are: "length is a field, not a method" and "indexes are 0 to length-1" are the two facts every other part of this question depends on. F.10 and Unit 8.1.

Question 7: 2024/2025, Question Two, 3 marks

Question 8: 2024/2025, Question Two, 7.5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Two

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program that does the following:

- i. Prompts the user to input 10 values into an array
- ii. Implement two methods
 - a. A method to find, count and display the number of even
 - b. Another method to sum and display the odd numbers.
- iii. In the main method, use a for loop to find and display the maximum number.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing. You write the whole program.

Asked for Three deliverables: input of 10 values, **two** methods, and a maximum found **in main**.

The marks 7.5 marks split across the three numbered parts. Part iii says "in the main method" and "use a for loop", so a method call or a while loop there is not what was asked.

Watch Part ii a reads "find, count and display the number of even" and stops mid-phrase. It means even *numbers*. Answer the obvious intent and move on.

Taught
in

Units 6.2 to 6.6, and F.10.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 D · TEN VALUES, EVENS, ODD SUM, AND THE MAXIMUM · 7.5 MARKS

Read the verbs: **find, count, display** is three things, and the even method must do all three. And the maximum must be found **in main with a for loop**, not in a third method: the question says where to put it.

CODE · Q2 D, COMPLETE · VERIFIED ON THE JVM

```
import java.util.Scanner; public class ArrayStats {    // ii a. find, count and display t
```

OutputEven numbers: 4, 12, 0, -8, 6

How many even: 5

Sum of the odd numbers: 39

Maximum: 15

Input is a Scanner here so the run can be printed. The ten values fed in were 4, 7, -3, 12, 9, 0, 15, -8, 6, 11.

MUST MEMORISE · TEST FOR ODD WITH `!= 0`, NEVER `== 1`

`if (v % 2 != 0)` is odd. **Always.**

`if (v % 2 == 1)` is odd **only for positives**, and silently wrong for negatives.

In Java, `%` keeps the sign of the **left** operand, so `-3 % 2` is **-1**, not 1. So `-3 % 2 == 1` is false and the `== 1` test declares -3 even.

The test data above proves it. The odd numbers are 7, -3, 9, 15 and 11, and they sum to **39**. Had the method used `== 1`, it would have skipped -3 and printed **42**: a wrong answer, from a program that runs cleanly and looks right. That is why the ten values include a negative odd number, and why yours should too. Unit 3.6 and Unit 8.4.

MEMORY HOOK: "not even" is safe; "equals one" is not.

TEACH · PRIMING THE MAXIMUM: WHY NOT START AT 0

The maximum loop starts `int max = values[0]`; and then runs from `i = 1`. The tempting alternative is `int max = 0`; and it is a bug waiting for the right data.

If every value is negative, `max = 0` never gets replaced and the program reports a maximum of **0**, a number that **is not in the array**. Priming with `values[0]` cannot do that: the answer starts as a real element and only ever moves to another real element.

The same reasoning as Unit 8.4's minimum, and the same reasoning that makes `Integer.MIN_VALUE` the other correct prime. Starting at 0 is correct only if you know the data contains a non-negative

number, and "I know my data" is how bugs are born.

Question 9: 2024/2025, Question Five, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Five

THE COURSE TEXT, UNEDITED

[3 marks]B. Write a method called `countNegatives` that accepts a single-dimensional array of integers and returns the number of negative values in the array. You do not need to write the full class or main method, just the method definition.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

- Given Nothing, but the signature is dictated to you in words.
- Asked **One method, and only the method.** The paper explicitly tells you not to write the class for or main.
- The marks 3 marks. Writing a full program wastes minutes and earns nothing extra.
- Watch Read the signature out of the sentence before you write: name `countNegatives`, takes `int[]`, returns `int`. Negative means `< 0`, so zero is not negative.
- Taught in Units 5.3 to 5.5 and 6.3.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 B · WRITE COUNTNEGATIVES · 3 MARKS

Read that concession and take it. Writing a whole class here wins no extra marks and costs you minutes you need for part D.

CODE · Q5 B · VERIFIED ON THE JVM

```
public class CountNegatives {    // Q5 B: the method the question asks for, and nothing
```

```
Output{3, -1, 0, -7, 8, -2} -> 3
{1, 2, 3} -> 0
{} (empty) -> 0
```

Only `countNegatives` is the answer; `main` is here so the method can be run. **Zero is not negative**, which is why the first array has a 0 in it and the count is 3, not 4: the test is `< 0`, never `<= 0`. And the empty array returns 0 without any special case, since the loop simply never runs.

Part 2: the method behind these, taught from nothing

Learn by Doing: Recursion

Topic Eight

Part B: the past questions on this topic, from both papers

Question One: 2024/2025, Question Six, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Six

THE COURSE TEXT, UNEDITED

[3 marks]B. What will be the return value of `compute(13)`? Show the steps to explain your answer.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A method and one argument.

Asked The return value **and the steps**. The instruction "show the steps" makes the trace part of the answer, not optional working.

The marks 3 marks. A bare number, even a correct one, is not a full answer here.

Watch Recognise what the method is: it counts the 1 bits in the binary form of `x`. Trace the table anyway, because that is what the marks are for.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 B · WHAT DOES COMPUTE(13) RETURN? SHOW THE STEPS · 3 MARKS

The answer: 3. And the question says **show the steps**, so the table is the answer, not the number.

DRY RUN · COMPUTE(13), STEP BY STEP

`while (x > 0): count the odd steps, halve each time`

x	x % 2	odd, so count++?	x = x / 2	count
13	1	yes	6	1
6	0	no	3	1
3	1	yes	1	2
1	1	yes	0	3
0	x > 0 is false, so the loop ends			3

What the method actually is. Repeatedly halving with `x / 2` and testing `x % 2` walks the number's **binary digits**, from the right. So `compute` counts the **1 bits**. 13 in binary is **1101**, which has three 1s, so the answer is 3, and you can check it in one second without tracing anything.

Say that in your answer after showing the table. Naming what the code *is* ("it counts the set bits") is what separates a trace from understanding, and it costs one line.

CODE · THE PAPER'S OWN METHOD, RUN ON 13 AND ON FRIENDS · VERIFIED ON THE JVM

```
public class ComputeBits {    // exactly as the paper prints it    public static int com
```

```
Outputcompute(13) = 3
```

```
13 in binary is 1101
```

```
compute(1) = 1    binary 1
```

```
compute(2) = 1    binary 10
```

```
compute(3) = 2    binary 11
```

```
compute(7) = 3    binary 111
```

```
compute(8) = 1    binary 1000
```

```
compute(255) = 8    binary 11111111
```

Every row is its own proof: the answer always equals the number of 1s printed beside it.

`compute(255) = 8` and 255 is eight 1s; `compute(8) = 1` and 8 is **1000**. The claim is not "this looks like a bit count", it is a bit count.

Part 2: the method behind these, taught from nothing

Learn by Doing: Exceptions and File Input and Output

Topic Nine

Part B: the past questions on this topic, from both papers

Question One: 2025/2026, Question One, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, Question One

THE COURSE TEXT, UNEDITED

[3 Marks]B. State three differences between checked exceptions and unchecked exceptions in Java

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Nothing.

Asked for Exactly **three** differences. Not a description of each kind: differences, stated as contrasts.

The marks 3 marks, one per difference. A fourth earns nothing, so spend the time elsewhere.

Watch Write them as pairs ("checked ... whereas unchecked ..."). A list of facts about checked exceptions alone is not a difference and markers do not award it.

Taught in Units 9.2 and 9.3.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 B · THREE DIFFERENCES: CHECKED VS UNCHECKED · 3 MARKS

Model answer. Three differences, one mark each:

1. **When they are detected.** Checked exceptions are checked by the compiler at **compile time**; unchecked exceptions occur at **run time** and the compiler does not examine them.
2. **Whether you must handle them.** A checked exception **must** be either caught in a try-catch or declared with throws, or the code **will not compile**. An unchecked exception carries no such obligation.
3. **What causes them.** Checked exceptions represent conditions outside the program's control (a missing file, a failed network call), whereas unchecked exceptions represent **programming errors** (dereferencing null, a bad index, dividing by zero).

Examples (add them; they cost one line and prove the point): checked: `IOException`, `FileNotFoundException`, `ClassNotFoundException`. Unchecked: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.

Where the marks are: three *distinct* axes. Writing "one is checked and one is not" three different ways scores one mark. See Unit 10.1.

Question Two: 2025/2026, The file question, 13 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2025/2026, The file question

THE COURSE TEXT, UNEDITED

(13 marks)B. A college admin office needs a program to record and review student details (ID, name, CGPA) using basic file I/O. Using only `java.io.FileWriter`, `java.io.FileReader`, `java.io.BufferedReader`, `java.io.BufferedWriter`, and `java.io.IOException` (or its subclasses), write a complete program that does the following:

i. Create a file named `input.txt` and write the details (student ID, name, CGPA) of at least three students into it using `FileWriter`, one student per line. Wrap the writing logic in a try-catch block that handles any exception that may occur.

ii. Read student details from a file named `output.txt` using `FileReader` and print each line to the console exactly as it appears in the file. Handle any exception that may occur while reading, including the case where `output.txt` does not exist.

iii. Create a method that takes in a file name, opens the file, counts the number of lines (student records) it contains, and returns that count. This method must handle its own exceptions internally.

iv. Ensure every file resource opened is properly closed (either in a finally block or via try-with-resources) and that every catch block prints a clear, specific message describing what went wrong.

Console output (System.out) is acceptable throughout.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A closed list of classes you are allowed to use.

Asked for A complete program, four numbered requirements, each mark-bearing.

The marks 13 marks. Requirement iv applies to **all** of the code above it, not just to part iii.

Watch It writes **input.txt** and reads **output.txt**. Those are different files, and part ii explicitly wants the "does not exist" case handled, so the mismatch is deliberate. Do not quietly read back the file you just wrote.

Taught in Units 10.2 to 10.7 and 9.2 to 9.5.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **13 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 B · STUDENT RECORDS WITH BASIC FILE I/O · 13 MARKS

1. creates **input.txt** and writes at least three students (ID, name, CGPA) into it with `FileWriter`, one per line, wrapped in try-catch;
2. reads student details from `output.txt` with `FileReader` and prints each line exactly as it appears, handling any exception **including the case where output.txt does not exist**;
3. has a method that takes a file name, counts the lines, and returns the count, **handling its own exceptions internally**;
4. closes every resource, and prints a clear, specific message in every catch block.

MUST MEMORISE · IT WRITES INPUT.TXT AND READS OUTPUT.TXT

Part 1 writes **input.txt**. Part 2 reads **output.txt**. The program never creates `output.txt`.

That is not a misprint for you to tidy up. It is the question.

Read part 2 again: "handle any exception that may occur while reading, **including the case where output.txt does not exist.**" The examiner has told you, in the question itself, that the file will be missing. The marks are for the **catch block running**, not for the file being read.

So do not "fix" it by reading input.txt instead. Write exactly what the paper asks, and let the catch block do its job. That is the answer.

MEMORY HOOK: the missing file is the exam. When a question names a file it never created, the catch block is the thing being marked.

TEACH · THE FIVE CLASSES, AND WHY THE LIST IS EXACTLY THESE FIVE

The restriction looks arbitrary. It is not: the five are exactly what you need and nothing more (Unit 10.3).

Class	Direction	What it is for
FileWriter	out	Writes characters to a file. On its own it writes one character at a time.
BufferedWriter	out	Wraps a FileWriter and collects characters, writing them in blocks. Also gives you <code>newLine()</code> .
FileReader	in	Reads characters from a file. On its own it reads one character at a time: it has no readLine() .
BufferedReader	in	Wraps a FileReader and gives you <code>readLine()</code> , which is the whole reason it is on the list.
IOException	-	The checked exception every one of the above can throw. You must catch it or declare it (Unit 10.1).

The pairing is the point. The question says "read using FileReader" and "one student per line". A FileReader alone cannot give you a line. So you wrap it: `new BufferedReader(new FileReader(fileName))`. That is why both appear on the list, and it is why the list is not padding.

And read "or its subclasses" carefully. Those three words are permission.

`FileNotFoundException` is a subclass of `IOException`, so you are explicitly allowed to catch it separately, which is exactly how you answer part 2's "including the case where output.txt does not exist" with a message that says what actually happened.

TRAP · CATCH THE CHILD BEFORE THE PARENT, OR IT WILL NOT COMPILE

When you catch both, the **subclass must come first**. Java tries catch blocks top to bottom and takes the first that fits. Put `IOException` first and it swallows everything, leaving the `FileNotFoundException` block unreachable. Java does not let that pass:

```
import java.io.BufferedReader;import java.io.FileNotFoundException;import java.io.FileRe
Compiler outputerror: exception FileNotFoundException has already been caught
```

This is one of Java's friendlier errors: it refuses to compile code containing a block that could never run. Reverse the two catch blocks and it compiles. **Specific first, general last**, every time.

CODE · QUESTION THREE, PART B · VERIFIED ON THE JVM

```
import java.io.BufferedReader;import java.io.BufferedWriter;import java.io.FileNotFoundException
```

```
OutputWrote 3 student records to input.txt
```

```
--- part ii: reading output.txt, exactly as the paper asks ---
```

```
Cannot read output.txt: the file does not exist.
```

```
--- the same method on the file we did write ---
```

```
STU001, Ada Bello, 4.51
```

```
STU002, Femi Okonkwo, 3.78
```

```
STU003, Ngozi Eze, 4.92
```

```
--- part iii: counting records ---
```

```
input.txt holds 3 records
```

```
Cannot count output.txt: the file does not exist.
```

```
output.txt holds -1 records
```

Line 3 of that output is the 13 marks. The program did exactly what the paper asked, the file was not there, and the catch block said so in words a human can read.

TEACH · FOUR DECISIONS IN THAT PROGRAM WORTH DEFENDING

1. try-with-resources answers part 4 on its own. Writing `try (BufferedWriter out = ...)` closes the writer automatically, whether the block ends normally or throws (Unit 10.2). The paper allows a `finally` block instead, and it is worth knowing why the bracket form is better: a `close()` inside `finally` can itself throw, and if it does while an earlier exception is already travelling, the second exception **replaces** the first and the real cause disappears. If you do use `finally`, the `close` must sit inside its own try-catch, and it must test for null, because the resource may have failed to open at all.

2. Wrapping means closing one thing, not two. `out.close()` on the `BufferedWriter` closes the `FileWriter` inside it. Closing the wrapper closes what it wraps, so one bracket handles both.

3. countRecords returns -1, not 0, when it fails. The method must "handle its own exceptions internally", so it cannot throw, so it must return something. Return 0 and the caller cannot tell a missing file from an empty one. **-1 is impossible as a line count**, so it can only mean failure. A value chosen because it cannot be mistaken for a real answer is called a **sentinel**, and picking a good one is a real skill.

4. Read the last two lines of the output in the right order. "Cannot count output.txt" prints **before** "output.txt holds -1 records", even though the second is the `println` that contains the first as an argument. Java **evaluates the arguments before it calls the method**: `countRecords` runs to completion, printing its own message, and only then does the outer `println` receive -1 and print. Argument first, call second, always.

Question Three: 2024/2025, Question Three, 3 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Three

THE COURSE TEXT, UNEDITED

[3 marks]A. Define the following: `PrintWriter`, `Files.write()` method and `try-with-resources` block.

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given	Nothing. Recall.
Asked for	Three definitions, one per term.
The marks	3 marks, one each.
Watch	All three are about writing and closing . The examiner is checking you know that a resource must be closed and what closes it for you.
Taught in	Units 10.2 to 10.6.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **3 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 A · DEFINE `PRINTWRITER`, `FILES.WRITE()`, AND `TRY WITH RESOURCES` · 3 MARKS

Model answer.

1. **PrintWriter.** A character-output class that writes **formatted text** to a file or stream, giving you the same familiar methods as `System.out`: `print`, `println` and `printf`. It is a wrapper: you hand it a destination (a file name, or another `Writer`) and it adds the formatting.
2. **Files.write().** A static method on `java.nio.file.Files` that writes an entire byte array or collection of lines to a file **in a single call**, creating the file if needed and closing it for you. It is the one-line alternative to opening a writer, looping, and closing.
3. **try-with-resources.** A try with the resource declared in brackets, `try (PrintWriter out = ...) { ... }`, which **automatically closes** that resource when the block ends, whether it ends normally or by an exception. It removes the need for a `finally` block whose only job is closing. Unit 10.2.

Where the marks are: for each, say **what it is** and **what it saves you**. "`PrintWriter` writes formatted text" earns the mark; "`PrintWriter` is for files" does not distinguish it from `FileWriter`.

Question Four: 2024/2025, Question Three, 3 marks

Question Five: 2024/2025, Question Three, 4 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Three

THE COURSE TEXT, UNEDITED

[4 marks]C. What is the output of the data.txt file?

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given A program that writes to a file.

Asked What ends up **in the file**, not what appears on screen. On screen this program prints for nothing at all.

The 4 marks for a two-line file. They are for the exact characters: the spacing, the line break, marks and the rounding.

Watch Three separate traps in four lines: print does not break the line, println does, and %.2f rounds 89.456 rather than truncating it.

Taught in Units 10.3, 10.4 and 3.9.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **4 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 C · WHAT IS THE OUTPUT OF THE DATA.TXT FILE? · 4 MARKS

Note the question: not what the program prints on screen (it prints **nothing**), but what ends up **in the file**.

The answer:

data.txtAge: 25

Score: 89.46

Two lines. And three details carry the four marks.

MUST MEMORISE · PRINT, PRINTLN, PRINTF: THE THREE DIFFERENCES

`print("Age: ")` writes the text, **no line break**. So the 25 lands on the same line.

`println(25)` writes it **and a line break**. That break is what ends line 1.

`printf("Score: %.2f", 89.456)` formats, and adds **no line break** unless you write `%n`.

1. %.2f rounds, it does not truncate. 89.456 to two places is **89.46**, not 89.45. Unit 3.11.

2. The file has no trailing newline. `printf` did not add one, so the file ends immediately after the 6 of 89.46. The file is 21 bytes and its last character is not a line break.

3. Nothing appears on screen. Every call is on `pw`, not on `System.out`. A common wrong answer is to write the two lines as program output.

MEMORY HOOK: only `println` breaks the line. `print` and `printf` leave the cursor exactly where it stopped.

CODE · THE PAPER'S CODE, PLUS A DUMP OF THE FILE IT WROTE · VERIFIED ON THE JVM

```
import java.io.*;public class PrintWriterExample {    public static void main(String[] a
```

Output--- data.txt verbatim ---

Age: 25

Score: 89.46

--- as escaped bytes ---

Age: 25

Score: 89.46

length = 21

The paper's four lines are exactly as printed; everything after `pw.close()` is ours, reading the file back so that this book can show you what landed in it. **Length 21** is the proof of the trailing-newline point: "Age: 25" is 7 characters, "Score: 89.46" is 12, which is 19, and the two remaining bytes are the single line break in the middle. There is nothing on the end.

Those two bytes are worth a word. `println` uses the **platform's** line separator, which on Windows is carriage-return plus line-feed (two characters) and on Linux is line-feed alone (one). Same program, 21 bytes here and 20 there. The two printed lines are the answer either way; if you are asked for a byte count, say which platform.

Question 6: 2024/2025, Question Three, 7.5 marks

THE SCENARIO, AS THE COURSE TEXT PRINTS IT

2024/2025, Question Three

THE COURSE TEXT, UNEDITED

[7.5 marks]D. Write a program that does the following:

- i. Create a file named `students.txt` if it does not exist.

ii. Uses `PrintWriter` to write the following student records to the file in a formatted table

Name	Age	GPA
John	20	3.75
Lilian	21	3.95
Ken	19	3.65

iii. Use `printf()` for proper column alignment

iv. Ensure the file is appended to if it already exists.

v. Handle possible exceptions properly

BREAK IT DOWN FIRST

before you write a word

What is this question **actually** asking, what have you been given, and what do the marks tell you about how much to write? Decide, then check.

ANSWER

Given Three rows of data to write.

Asked Five numbered requirements, and **each one is a mark-bearing instruction**: create, write for with `PrintWriter`, align with `printf`, append, handle exceptions.

The marks 7.5 marks over five parts. Tick them off in your answer so the marker can too.

Watch Requirements iv and i pull against each other: **append** mode is what keeps existing content, and it is a constructor argument, not something you add later.

Taught in Units 10.3 to 10.6 and 3.9.

PART 1: THE ANSWER YOU WRITE IN THE HALL

write yours first, then open this

Write your answer to the question above in your book first. It carries **7.5 marks**, so write one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 D · WRITE STUDENTS.TXT AS A FORMATTED, APPENDED TABLE · 7.5 MARKS

MUST MEMORISE · `PRINTWRITER` CANNOT APPEND. THE `FILEWRITER` UNDER IT CAN

`new PrintWriter("students.txt")` **truncates** the file to empty. Every time. There is no append constructor.

`new PrintWriter(new FileWriter("students.txt", true))` appends.

The `true` is the append flag, and it belongs to **FileWriter**, not to `PrintWriter`. So the answer to "append with `PrintWriter`" is to **wrap** one around a `FileWriter` opened in append mode, exactly as Unit 10.3 wraps a `BufferedReader` around a `FileReader` to get `readLine()`. Each wrapper adds what the other lacks.

And it satisfies part i for free. `FileWriter` **creates the file if it does not exist**, in either mode. So "create `students.txt` if it does not exist" needs no `if`, no `File.exists()`, no check at all: opening it in append mode does both jobs.

MEMORY HOOK: the flag is on the writer that touches the disk. `PrintWriter` only formats; `FileWriter` opens the file, so the mode is its business.

CODE · Q3 D, COMPLETE · VERIFIED ON THE JVM

```
import java.io.BufferedReader;import java.io.FileReader;import java.io.FileWriter;import
students.txt, read back, every space shown as a dotName.....Age...GPA...
John.....20....3.75..
Lilian...21....3.95..
Ken.....19....3.65..
```

The dots are the marks. `%-8s` means: a String, left aligned, **padded to 8 columns**, and you can count the padding: "Name" plus four dots, "Lilian" plus two. The minus sign is what makes it left aligned; drop it and every column is pushed right and the table goes ragged. `%-6.2f` is the same idea for a double, fixed at 2 decimal places, which is why 3.75 prints as 3.75 and not 3.8. Unit 3.11.

Those trailing dots are real spaces, sitting at the end of every line. `printf` pads the last column just as faithfully as the others, because it cannot know it is last. Harmless in a text file, and worth knowing before you go looking for why two files that "look identical" are not.

CODE · PROOF THAT THE APPEND FLAG IS DOING THE WORK · RUN TWICE, IN ONE PROGRAM

The claim above is that the `true` is what makes it append and that without it the file is wiped. Here that claim is tested: the same write is done **twice** each way, and the lines are counted.

```
import java.io.BufferedReader;import java.io.FileReader;import java.io.FileWriter;import
Outputnew PrintWriter(new FileWriter(f, true)) twice -> 2 lines
new PrintWriter(f)                               twice -> 1 lines
```

Two lines against one. The appending version kept both records; the plain `PrintWriter` threw the first away and left only the second. If you write `new PrintWriter("students.txt")` in the exam, your program passes every other part of this question and still fails part iv, and nothing in its output tells you.

Part 2: the method behind these, taught from nothing

Mock Paper B, in the Current Shape

Mock paper

Question One

MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 A · DEFINE A CONSTRUCTOR · 2 MARKS

Model answer.

A **constructor** is a special member of a class that runs once, when an object of that class is created with `new`, and whose job is to put the new object's fields into a valid starting state.

The compiler tells it apart from an ordinary method by **two** facts, and both are needed for the mark: its **name is exactly the class name**, and it declares **no return type at all**, not even `void`. Those are not two ways of saying the same thing. Write `void MeterAccount(...)` and you have not written a constructor: you have written an ordinary method that happens to be called `MeterAccount`, it compiles, it is never called by `new`, and your fields stay at their defaults.

Where the marks are: one for what it is for, one for how it is recognised. "It has the same name as the class" alone is half the rule and often half the mark.

Q1 B · THREE DIFFERENCES: WHILE AGAINST DO WHILE · 3 MARKS

A three-row table beats a paragraph under time pressure, and it is easier to mark, which is the same thing.

	while	do-while
When it tests	Before the body: it is a pre-test loop.	After the body: it is a post-test loop.
Fewest runs	Zero. If the condition is false at the start, the body never runs.	One. The body always runs at least once, whatever the condition.
Syntax	No semicolon after the closing brace.	Ends with <code>while (condition);</code> and that semicolon is required .

Where the marks are: one per row. The middle row is the one that decides which to use: "must this run at least once?" is the only question that matters, and it is why input validation is usually a `do-while`.

Q1 C · FIND THE ERRORS · 5 MARKS

Five errors, one per mark, and they are the **same five kinds** that both real papers plant. Run the TTOIS checklist from S.1 and they fall out in about a minute.

1. **T for Type. Line 4:** `int[] arr` cannot hold `15.0`. The required output is **93.0**, with a point, which is the question telling you the array is `double[]`. Fix: `double[] arr`.
2. **T for Total. Line 5:** `int sum` cannot accumulate decimals. Fix: `double sum`.
3. **O for Off-by-one. Line 8:** `i <= arr.length` would reach `arr[6]` in an array of 6. Fix: `i < arr.length`.
4. **I for Inverted. Line 10:** `if (arr[j] % 3 == 0) continue;` skips **exactly the multiples of 3 it was told to find**. Fix: `% 3 != 0`.
5. **S for Stop. Line 14:** `if (matches == 4)` stops after four, and the specification says three. Fix: `matches == 3`.

What the compiler actually said: errors 1 and 2 only, as six repeats of "incompatible types: possible lossy conversion from double to int". Errors 3, 4 and 5 are **not compile errors at all**. Fix the types and this program compiles, runs, and prints a confident wrong answer. That is 3 of the 5 marks sitting in code the compiler is perfectly happy with.

CODE · Q1 C CORRECTED, ALL FIVE FIXES MARKED · VERIFIED ON THE JVM

```
// JOptionPane replaced by println so the output can be printed here;// in the exam, keep t
OutputSum: 93.0
Matches: 3
```

Check it by hand, because the examiner gave you the answer and expects you to work back to it. The multiples of 3 are 15, 36, 54 and 60. Starting at `i=0` the first one found is 15, giving 17. At `i=1` it is 36, giving 38. At `i=2` it is 36 again, giving 38, and that is the third match, so it stops. $17 + 38 + 38 = \mathbf{93.0}$, and **3** matches.

Q1 D I · THE FIVE OUTPUTS · 10 MARKS, PART ONE

Square it, then keep as many digits off the right-hand end as the number itself has, and compare.

n	digits	n * n	power = 10^digits	tail = square % power	output
5	1	25	10	25 % 10 = 5	AUTOMORPHIC
6	1	36	10	36 % 10 = 6	AUTOMORPHIC
7	1	49	10	49 % 10 = 9	NOT AUTOMORPHIC
25	2	625	100	625 % 100 = 25	AUTOMORPHIC
76	2	5776	100	5776 % 100 = 76	AUTOMORPHIC

`% power` is the whole trick: `% 10` keeps the last digit, `% 100` keeps the last two. "Ends in" is a remainder, and that is the idea the question is testing.

CODE · Q1 D, THE ORIGINAL, RUN FOR REAL · VERIFIED ON THE JVM

```
public class AutomorphicChecker {    public static void main(String[] args) {        int[] r
```

```
Output5 -> AUTOMORPHIC
6 -> AUTOMORPHIC
7 -> NOT AUTOMORPHIC
25 -> AUTOMORPHIC
76 -> AUTOMORPHIC
```

MUST MEMORISE · LOOP CONVERSION IS MECHANICAL. MOVE THE THREE PARTS, CHANGE NOTHING ELSE

A for is a while with its three parts collected onto one line:

```
for (INIT; TEST; UPDATE) { BODY }
```

is exactly

```
INIT; while (TEST) { BODY UPDATE }
```

So the conversion has three moves and no thinking: **INIT** goes above the loop, **TEST** stays in the brackets, **UPDATE** goes to the bottom of the body. Going the other way, collect all three back onto the header line. A for with an empty init and update, `for (; temp > 0; )`, is a legal and honest way to write a While as a For.

MEMORY HOOK: INIT up, UPDATE down. Ten marks on the real paper and there is no idea in it at all, only care.

CODE · Q1 D II, CONVERTED: THE FOR BECAME A WHILE, THE WHILES BECAME FORS · VERIFIED ON THE JVM

```
public class AutomorphicConverted {    public static void main(String[] args) {        int[]
```

Output, identical to the original5 -> AUTOMORPHIC

```
6 -> AUTOMORPHIC
7 -> NOT AUTOMORPHIC
25 -> AUTOMORPHIC
76 -> AUTOMORPHIC
```

The output is byte for byte the output of the original, and that is the entire test. A

conversion that changes the answer is not a conversion. Both versions were run and their outputs compared, which is exactly what you should do on paper: trace one line of each and check they agree.

TRAP · THE UPDATE MUST SURVIVE THE MOVE, ESPECIALLY PAST A CONTINUE

The commonest way to lose these ten marks is to move INIT up and then forget UPDATE, leaving `while (idx < numbers.length)` with no `idx++` anywhere. It compiles. It hangs forever. The marker sees an infinite loop where a conversion was asked for.

And there is a sharper version, worth knowing because "INIT up, UPDATE down" is **not always safe**: it breaks the moment the body contains a `continue`. A for runs its update on every way round, including the ones `continue` cuts short. A while has no update slot, so the one you moved to the bottom of the body gets **jumped over**, and the loop stops advancing.

Both halves were compiled and run. This for, summing the odd numbers below 6 by skipping the even ones with `continue`, prints **total = 9**. Converted to a while by the rule above, with `i++` at the

bottom of the body, it **hangs forever**: at `i = 0` the `continue` fires, `i++` is never reached, and `i` is 0 for eternity.

Neither loop in this question contains a `continue`, so the rule is safe here. **Check that before you apply it.** If the body does have one, the update must be repeated before the `continue` as well, or the loop must stay a `for`.

Question Two

MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 A · DEFINE INHERITANCE · 2 MARKS

Model answer.

Inheritance is the OOP principle by which one class, the **subclass** or child, acquires the fields and methods of another, the **superclass** or parent, using the `extends` keyword. The subclass gets everything the superclass has that it is allowed to see, and may then **add** members of its own or **override** inherited methods to behave differently.

Its purpose is to express an **is-a** relationship and to reuse code without copying it: a `PrepaidMeter` **is a** `MeterAccount`, so it needs no second copy of `topUp`. Because it is an is-a relationship, a subclass object may be used anywhere the superclass is expected, which is what makes runtime polymorphism possible.

Where the marks are: one for the mechanism (`extends`, and the subclass acquires the members), one for the point of it (reuse, and the is-a relationship). Note what inheritance does **not** give you: a subclass inherits a private field in the sense that the object carries it, but it cannot name it. That distinction is Question Four.

Q2 B · THREE DIFFERENCES: AN ARRAY AGAINST A VARIABLE · 3 MARKS

	An ordinary variable	An array
How many values	Exactly one, of its declared type.	Many, all of the same type, reached by index.
What the name holds	The value itself, for a primitive: <code>int x = 5</code> holds 5.	A reference to an object elsewhere in memory. Declaring <code>int[] a;</code> creates no array at all, and <code>a</code> is <code>null</code> until <code>new</code> makes one.
Size	Not a question you can ask.	Fixed at creation and readable through <code>length</code> , which is a field , not a method: <code>a.length</code> , never <code>a.length()</code> .

Where the marks are: one per row. The second row is the one that earns respect, because it is the reason two array names can refer to the same array, and the reason passing an array to a method

lets that method change it (Unit 5.4).

Q2 C · FIND THE ERRORS · 5 MARKS

The same five kinds again. **TTTOS**: Type, Total, Off-by-one, Inverted, Stop.

1. **T. Line 4:** `int[] weights` cannot hold 3.5. The required total, **84.6**, has a point in it. Fix: `double[] weights`.
2. **T. Line 5:** `int total` cannot accumulate decimals. Fix: `double total`.
3. **O. Line 8:** `i <= weights.length`. Fix: `i < weights.length`.
4. **I. Line 9:** `if (weights[i] > 5) continue;` skips **exactly the parcels it was told to charge** and charges the ones it was told to skip. Fix: `if (weights[i] < 5) continue;`.
5. **S. Line 15:** `if (count == 5)` stops after five, and the specification says four. Fix: `count == 4`.

What the compiler actually said: errors 1 and 2, as six repeats of "incompatible types". Once again the three interesting errors compile perfectly.

CODE · Q2 C CORRECTED, ALL FIVE FIXES MARKED · VERIFIED ON THE JVM

```
// JOptionPane replaced by println so the output can be printed here;// in the exam, keep t
```

```
OutputTotal: 84.6
```

```
Count: 4
```

Work back to it: the weights of 5 and above are 12.0, 20.0, 8.5 and 30.0, which is four, so the stop-at-four fires on the last one. With a 20% surcharge they cost 14.4, 24.0, 10.2 and 36.0, and those add to **84.6**.

TRAP · "SKIP ANYTHING UNDER 5" IS NOT "CONTINUE IF OVER 5"

Error 4 is the one this lecturer plants every year, in both questions, in both papers, and it is worth understanding rather than memorising. The English says **skip the small ones**. The code says `if (weights[i] > 5) continue;`, which skips the **big** ones. It is not a typo for `<`: it is the condition **inverted**, and inverted code reads perfectly.

The defence is mechanical. Whenever you see `continue`, say out loud: "**this line names the ones I am throwing away.**" Then check that sentence against the specification. Here the specification throws away the ones under 5, so the condition after `if` must be the under-5 test, and it is not.

Q2 D I · THE OUTPUT · 10 MARKS, PART ONE

Prime: 10, Composite: 19, Neither: 1

The inner loop counts **divisors**, testing every `d` from 1 to `num` inclusive, so it counts 1 and `num` themselves.

- **1** has exactly one divisor, itself. Not 2, not more than 2, so it falls to the `else`: **Neither: 1**.
- **Primes** up to 30 are 2, 3, 5, 7, 11, 13, 17, 19, 23, 29: **10** of them.
- Everything else is composite: 30 numbers, minus 1, minus 10, is **19**.

The check that costs nothing: **10 + 19 + 1 = 30**, and the loop ran 30 times. Every number landed in exactly one bucket, so no number was counted twice or dropped. Do that sum in the hall.

CODE · Q2 D, THE ORIGINAL, RUN FOR REAL · VERIFIED ON THE JVM

```
public class PrimeClassifier {    public static void main(String[] args) {        int prime;
OutputPrime: 10, Composite: 19, Neither: 1
```

CODE · Q2 D II, BOTH FOR LOOPS CONVERTED TO WHILE LOOPS · VERIFIED ON THE JVM

```
public class PrimeClassifierWhile {    public static void main(String[] args) {        int ;
Output, identical to the originalPrime: 10, Composite: 19, Neither: 1
```

Both loops convert, and the inner one is where the marks are lost. Its `int d = 1` must be reset **inside** the outer loop, once per num, because that is where the for version's init lived. Hoist it above the outer loop, as people do when they are converting in a hurry, and `d` keeps the value it reached on the previous number. It was compiled and run: the answer becomes **Prime: 0, Composite: 0, Neither: 30**. Every number falls into the same bucket, because the inner loop now runs at most once and finds one divisor every time. Same output as the original, or the conversion is wrong.

Question Three

MODEL ANSWER

Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 A · READ THE DEFINITION BEFORE YOU WRITE A LINE · 12 MARKS

Everything here is decided by two words in the question: **strictly**, and **both**. A spike beats **both** neighbours, **strictly**. Get those into the condition and the program is fifteen lines.

The hint is the specification restated, and it is worth trusting: the answer must not flag 12, 22, 27, 27 or 25. Work out why each is excluded **before** you code, because each one is excluded for a different reason.

Index	Reading	Neighbours	Verdict
0	12	only one, on the right	never checked: an end
1	30	12 and 18	SPIKE : beats both
2	18	30 and 22	DIP : below both
3	22	18 and 41	above 18 but below 41: neither
4	41	22 and 27	SPIKE : beats both
5	27	41 and 27	below 41, but equal to 27: not strictly below both
6	27	27 and 33	equal to 27, below 33: neither
7	33	27 and 19	SPIKE : beats both
8	19	33 and 25	DIP : below both

9	25	only one, on the left	never checked: an end
---	----	-----------------------	-----------------------

So: 3 spikes, and 5 flagged readings: 30, 18, 41, 33, 19.

CODE · QUESTION THREE, PART A · VERIFIED ON THE JVM

```
public class SpikeReport {    // i. ONE pass through the array: how many readings beat both
OutputSpikes: 3
Flagged: 30, 18, 41, 33, 19
```

Both methods loop $i = 1$ to `readings.length - 2`, and that is the whole answer to "the ends are never spikes": do not test them, rather than test them and guard. $i - 1$ and $i + 1$ are then always in bounds, by construction, and no `ArrayIndexOutOfBoundsException` is possible. `println` here so the run can be printed; in the hall it is one `showMessageDialog` carrying both lines (F.11).

TRAP · THE EQUAL PAIR AT INDEXES 5 AND 6 IS THE WHOLE QUESTION

The readings 27 and 27 sit side by side, and they are there on purpose. Write `>=` where the question said **strictly** greater and index 6 becomes a spike, because `27 >= 27`. Write `<=` for the dip and index 5 becomes a dip, for the same reason. Every other reading in the array answers identically either way.

That is what a planted equal pair is for. It is the only element of the array that can tell a marker whether you read the word "strictly" or skimmed past it. This lecturer plants one in every array question: the 2025/26 Question Three does the same with its consecutive duplicates.

MUST MEMORISE · BUILDING A COMMA-SEPARATED STRING WITHOUT A TRAILING COMMA

```
out = out.equals("") ? "" + value : out + ", " + value;
```

The separator goes **before** every item except the first, so ask "am I first?" instead of trimming a comma off the end afterwards.

The obvious way, `out = out + value + ", ";`, leaves 30, 18, 41, 33, 19, with a comma and a space hanging off the end, and the marker can see it. The obvious repair, chopping the last two characters off, breaks when the list is empty and there are no two characters to chop.

The empty check is not decoration: it is the base case. And note `"" + value`, which is how you turn an `int` into a `String` with no method call at all.

MEMORY HOOK: the comma belongs to the item on its left. So the first item does not get one, and no item ever gets one it has to give back.

Q3 B · THE RESTRICTED API LIST IS THE ANSWER PLAN · 13 MARKS

The question names five classes and permits nothing else: `FileWriter`, `FileReader`, `BufferedReader`, `BufferedWriter`, `IOException`. Reach for `PrintWriter` or `Files.write()` here and you lose marks for code that works, because the restriction **is** the question.

Read part ii carefully: it says `read output.txt`, and nothing in this program ever creates `output.txt`. That is not an oversight in the paper. **The missing file is the test**, and your catch block firing is the right answer, not a bug.

CODE · QUESTION THREE, PART B · VERIFIED ON THE JVM

```
import java.io.BufferedReader;import java.io.BufferedWriter;import java.io.FileReader;import java.io.FileWriter;

Output, run in a folder with no output.txtinput.txt written
Could not read output.txt: it does not exist yet.
input.txt holds 3 records
Could not count output.txt: it could not be opened.
output.txt holds 0 records
```

Read the last two lines together. The counting method met a file that does not exist, **handled it itself**, said so, and returned 0. It did not throw, and main was never asked to have an opinion. That is exactly what part iii means by "must handle its own exceptions internally", and it is why the method returns an int and not a wobble.

TRAP · DO NOT PRINT E.GETMESSAGE() AND CALL IT A SPECIFIC MESSAGE

Part iv asks every catch block to print "a clear, specific message describing what went wrong", and the reflex is `System.out.println(e.getMessage())`. Resist it. For a missing file the JVM fills that message with the **operating system's own words**: on the machine this book was built on it reads `output.txt (The system cannot find the file specified)`, while on a different system it reads `output.txt (No such file or directory)`.

So the same program prints different text on different machines, and neither sentence says what **your program** was trying to do when it failed. "Could not count output.txt" does. Name the operation and the file; let the exception object stay in the log where it belongs.

MUST MEMORISE · TRY-WITH-RESOURCES CLOSES IT, ON EVERY PATH, INCLUDING THE ONE YOU FORGOT

```
try (BufferedReader in = new BufferedReader(new FileReader(f))) { ... }
```

closes in when the block ends, **whether it ends normally or by throwing**. No finally, no `close()`, nothing to forget.

Part iv accepts a finally block, and a finally is a correct answer. But it is three more lines, and its own `close()` can itself throw, so the honest version needs a try inside the finally. The resource form is shorter **and** harder to get wrong, which is the only kind of shorter worth having.

MEMORY HOOK: the brackets after try own the thing inside them. Whatever is declared there is closed when the block ends, in the reverse of the order it was opened.

Question Four

MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 · WHAT THE THREE PARTS ARE REALLY ASKING · 25 MARKS

The marks split 9 / 10 / 6, and the shape of the answer is dictated:

- **Part 1 (9):** a superclass with **private** fields. The word private is doing real work and it is examined in Part 2.
- **Part 2 (10):** two subclasses, each **overriding the same method** differently. This is where the marks are.
- **Part 3 (6):** a main that **proves** the overrides run, including through a superclass reference.

Part 3 v is the sentence that decides your grade: "**confirming that the correct overridden version runs**". A main that calls `pre.consume(...)` and `post.consume(...)` directly does not confirm anything: of course they run their own methods. The confirmation is calling it through a `MeterAccount` reference, where the compiler cannot know which one it will be.

CODE · QUESTION FOUR, ALL THREE PARTS · VERIFIED ON THE JVM

```
// Part 1: the superclass, with PRIVATE stateclass MeterAccount {    private String meterNur

Output--- before ---
Meter PRE-001 | Ada | 50.0 units
Meter POST-002 | Bola | 50.0 units
PRE-001 topped up 20.0 units. Balance: 70.0
POST-002 topped up 20.0 units. Balance: 70.0
--- transactions ---
PRE-001 used 30.0 units. Balance: 40.0
PRE-001 PREPAID REJECTED: 38.0 units would leave less than the 5.0 unit reserve.
POST-002 used 30.0 units. Balance: 40.0
POST-002 POSTPAID rebate: 1.0 unit added. Balance: 41.0
POST-002 REJECTED: 100.0 units requested, only 41.0 available.
--- after ---
Meter PRE-001 | Ada | 40.0 units
Meter POST-002 | Bola | 41.0 units
--- through a MeterAccount reference ---
PRE-001 used 10.0 units. Balance: 30.0
POST-002 used 10.0 units. Balance: 31.0
POST-002 POSTPAID rebate: 1.0 unit added. Balance: 32.0
Meter PRE-001 | Ada | 30.0 units
Meter POST-002 | Bola | 32.0 units
```

Read the last block. One call site, `meters[i].consume(10.0)`, inside one loop, over an array whose type is `MeterAccount`. The prepaid meter checked its reserve; the postpaid meter paid its rebate. The compiler could not have chosen either: **the objects did**.

MUST MEMORISE · A SUBCLASS CANNOT SEE A PRIVATE FIELD. THAT IS NOT A BUG IN YOUR CODE

Part 1 says **private** fields. So `PrepaidMeter` may **not** write units. It must go through `getUnits()` and `setUnits()`.

Write `units - amount >= 5.0` inside the subclass and it is a compile error: `units` has private access in `MeterAccount`.

The object **has** the field: a `PrepaidMeter` genuinely carries its own units, and `topUp` changes it. The subclass simply cannot **name** it. Inheritance gives you the state; it does not give you permission.

This is the one place where "just make it protected" is the wrong instinct in an exam: the question **said** private, so the accessor is the answer, and reaching for protected to dodge the compiler is answering a question nobody asked.

MEMORY HOOK: private means private to the class, not to the object. The child owns the box and still cannot read the label.

TRAP · THE OVERRIDE MUST CALL SUPER, OR YOU HAVE REWRITTEN THE PARENT BADLY

Both overrides here do their own check and then hand the actual work to `super.consume(amount)`. That is deliberate, and it is worth a mark you will not see itemised.

Rewrite the deduction inside `PrepaidMeter` instead, and you now have two copies of "take the units off and print the balance". They agree today. They will not agree the moment anything changes, and the marker has seen that answer a hundred times.

`PostpaidMeter` shows the sharper reason. It has to know whether the deduction **actually happened**, because a rejected one earns no rebate. It cannot know that by re-checking the balance rule, because that would be a third copy of it. So it reads the balance before, calls `super`, and reads it after. **Ask the parent what it did; do not re-derive it.**

RECALL AND CHECK

In the last block, `meters` is declared `MeterAccount[]`. So why does `meters[1].consume(10.0)` pay a rebate, when `MeterAccount.consume` contains no rebate at all?

ANSWER Because the method that runs is chosen by the **object**, at run time, not by the reference's type at compile time. `meters[1]` refers to a `PostpaidMeter` object, so `PostpaidMeter.consume` runs. The declared type `MeterAccount` only decides what the compiler will **let** you call; it does not decide which version answers. That is runtime polymorphism, and it is the entire point of Part 3 v.

Question Five

MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

MUST MEMORISE · SECTION C TELLS YOU WHICH CONSTRUCT TO USE, AND MARKS YOU ON IT

Section C does not merely ask for a working program. It **names the construct**, and using a different one loses the mark even when the output is right.

"Use a **switch**" · "use **nested** if-else (not a switch, and not independent/separate if checks)" · "using a nested control statement" · "no arrays and no user-defined helper methods" · "no collection classes" · "JOptionPane only" · "each task must be its own method".

And from the front page: use comments in Section B and Section C to say what each block does. That is an instruction, and it is free marks.

MEMORY HOOK: underline every noun that names a construct before you write anything. Those nouns are the marking scheme.

CODE · QUESTION FIVE, PART A · VERIFIED ON THE JVM

```
import java.util.Scanner; public class CourierPricing {    // Part i: a switch returning the
Output, for 5 parcels
Parcel 1: Gold, 3 zone(s), 25.0 kg, base 580.0, surcharge 5.0%, pays 56
Parcel 2: Gold, 1 zone(s), 10.0 kg, base 120.0, surcharge 0.0%, pays 120.00
Parcel 3: Silver, 2 zone(s), 30.0 kg, base 320.0, surcharge 15.0%, pays 368.00
Parcel 4: Bronze, 2 zone(s), 5.0 kg, base 320.0, surcharge 25.0%, pays 400.00
Parcel 5: Platinum, 3 zone(s), 5.0 kg, base 580.0, surcharge 25.0%, pays 500.00
SUMMARY
Total revenue: 1888.00
Gold: 2, Silver: 1, Bronze: 2
```

The test data is chosen so every rule fires visibly. Parcels 1 and 5 would both have paid more than 500 and both paid exactly 500.00, so the ceiling is working. Parcel 5 is "Platinum", which is not Gold and not Silver: it got the 25% Bronze surcharge **and** was counted as Bronze, so the two Bronze rules agree with each other. Input is a Scanner here so the run can be printed; in the hall every prompt is `JOptionPane.showInputDialog` and the summary is one `showMessageDialog` (F.11).

TRAP · IF PLATINUM IS PRICED AS BRONZE, IT MUST BE COUNTED AS BRONZE

The question says any tier that is not "Gold" or "Silver" is treated as Bronze. That rule has to be obeyed **twice**: once when pricing, and once when counting for the summary. It is easy to write the pricing branch correctly and then count the tiers with a separate chain that has a Platinum case, or that counts nothing at all for it.

Then your dialog reports Gold 2, Silver 1, Bronze 1, and four parcels in a summary of five. The counts must add up to N. **Check that they do before you put your pen down:** $2 + 1 + 2 = 5$, and there were 5 parcels.

MUST MEMORISE · A SWITCH THAT RETURNS NEEDS NO BREAK

```
case 1: return 120.0; has no break, and needs none.
```

`return` leaves the whole **method**, so it certainly leaves the switch. There is nothing to fall through to.

The default throwing `IllegalArgumentException` is what makes the method total: every zone either returns a price or raises an error, and none silently returns 0. That is the question's own instruction, and it is also what stops a bad zone from quietly costing nothing.

MEMORY HOOK: break or return, never neither, never both. A switch that assigns needs breaks; a switch that returns does not.

CODE · QUESTION FIVE, PART B · VERIFIED ON THE JVM

```
import java.util.Scanner; public class NoiseLog {    public static void main(String[] args) ·
Output, for a log with two invalid entries at hour 0
Hour 0: 999 is outside 0 to 140. Enter :
Hour 0: abc is not a number. Enter it again.
```

Qualifying streaks (3 or more hours): 2

Longest breach streak: 6 hour(s)

Classification: CRITICAL

The 20 readings after the two rejects are 80, 82, 90, 91, 88, 70, 86, 87, 90, 92, 95, 88, 60, 90, 91, 70, 71, 72, 73, 74. The breaches (above 85) form runs of **3** (90, 91, 88), then **6** (86, 87, 90, 92, 95, 88), then **2** (90, 91). So two runs reach 3 and qualify, the run of 2 does not, and the longest is 6, which is CRITICAL. Both rejects were for hour **0**, and the third value became hour 0: the loop re-prompts the **same** hour, which is what "re-prompt for the same hour" means.

MUST MEMORISE · COUNT A STREAK WHEN IT REACHES THE THRESHOLD, NOT WHEN IT ENDS

```
if (run == 3) streaks++;
```

Exactly == 3, not >= 3.

This single line is the elegant half of the question. A run of 6 passes through `run == 3` once, on its third hour, and is counted once. Write `>= 3` and that run of 6 is counted **four** times, once for each hour from the third onward, and the answer becomes 5 instead of 2.

The alternative is to wait until the run breaks and count it then, but that needs a second check after the loop for a streak still running at the last hour, and forgetting it is the classic bug. Counting at the threshold has no end case at all.

MEMORY HOOK: count it as it qualifies, not as it dies. The moment it crosses the line is unique; the moment it ends may never come.

TRAP · VALIDATION MUST CATCH BOTH KINDS OF BAD INPUT, AND ONE OF THEM IS NOT A NUMBER

"It must parse as a number between 0 and 140" is **two** requirements, and the marks are split. 999 is a perfectly good number that is out of range: an `if` catches it. `abc` is not a number at all:

`Double.parseDouble("abc")` **throws** `NumberFormatException`, and no `if` in the world catches that. It needs a `try-catch`.

A solution with only the range check compiles, passes every test you would think to run by hand, and dies the first time a marker types a letter. Both entries are in this book's test data on purpose, and the output above shows both messages, because an untested validation is a guess.

Question Six

MODEL ANSWER

Question Six

Write your full answer to **Question Six** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q6 A · THE CONSTRAINTS ARE HALF THE MARKS · 12 MARKS

Read what is forbidden: **no arrays, no helper methods**, everything **inline in main**, the digit loop built **only** on `%` and `/`, and the primality test limited to the **square root**. A beautiful, well-factored answer with four private static methods scores badly here, and that is not unfair: the question is

testing whether you can hold a whole algorithm in one scope, which is exactly what a dry run asks of you.

Both facts in step 1 come out of **the same single loop**, and step 2 rides along in it too. Three answers, one pass.

DRY RUN · N = 1237, ONE ROW PER TURN OF THE DIGIT LOOP

temp	digit	product	largest	digit > previous?	previous
1237	7	1 x 7 = 7	7	first digit: nothing to compare	7
123	3	7 x 3 = 21	7	3 > 7? no	3
12	2	21 x 2 = 42	7	2 > 3? no	2
1	1	42 x 1 = 42	7	1 > 2? no	1
0	temp > 0 is false: the loop ends. product 42, largest 7, monotone stayed true				

The loop walks **right to left**, because % 10 can only reach the last digit. So previous holds the digit to the **right** of the one in hand, and "never decreases left to right" becomes "each digit is not greater than the one to its right". Same fact, read from the end you can actually reach.

Then: is 42 prime? $d = 2$, and $2 * 2 \leq 42$, and $42 \% 2 == 0$, so no, immediately. N is monotone but its product is not prime, so the second branch fires: **MONOTONE**.

CODE · QUESTION SIX, PART A · VERIFIED ON THE JVM

```
import java.util.Scanner; public class DigitClassifier {    public static void main(String[]
```

Output, for the input 1237N = 1237

Digit product = 42

Largest digit = 7

Classification = MONOTONE

MUST MEMORISE · $D * D \leq N$ IS THE SQUARE ROOT TEST, AND IT NEEDS NO MATH

```
for (int d = 2; d * d <= product; d++)
```

is the same test as $d \leq \text{sqrt}(\text{product})$, with no `Math.sqrt`, no double, and no rounding to argue about.

Why the square root at all: if n has a divisor bigger than its square root, it must also have the matching one **below** it, because the two multiply to n. So a divisor above the root can never be the first one found, and testing past the root can never discover anything new.

Prefer $d * d \leq \text{product}$ to $d \leq \text{Math.sqrt}(\text{product})$. It stays in integers, so there is no question about whether the root of 49 comes back as 6.999999 and drops a divisor.

MEMORY HOOK: divisors come in pairs, and the root is the pivot. Every pair has one member on each side, so one side is enough.

CODE · Q6 A, EVERY CLASSIFICATION PROBED · VERIFIED ON THE JVM

```
import java.util.Scanner; public class DigitClassifierBench {    public static void main(Str:
```

```

Output113    product 3    largest 3  -> MONOTONE-PRIME-PRODUCT
1237    product 42    largest 7  -> MONOTONE
405     product 0     largest 5  -> ZERO-PRODUCT
4821    product 64    largest 8  -> STANDARD
7       product 7     largest 7  -> MONOTONE-PRIME-PRODUCT
1119    product 9     largest 9  -> MONOTONE

```

Not part of the answer: this is the same inline logic run over six numbers, so that **all four** classifications appear and the priority order is visible. 7 is the edge worth writing: a single digit, the loop runs once, and it is monotone by definition because there is no pair of digits to go the wrong way. 1119 shows that "never decreases" allows **equal** digits, which is why the test is `digit > previous` and not `>=`.

TRAP · THE DIGIT PRODUCT STARTS AT 1, AND A ZERO DIGIT IS WHY ZERO-PRODUCT EXISTS

Two initialisation traps sit next to each other, and they pull in opposite directions.

`int product = 1;`, not 0. A product accumulates by multiplying, and anything times 0 is 0, so starting at 0 makes every answer 0. Meanwhile `int largest = 0;` is right, because a largest accumulates by comparing and every digit is at least 0.

And 405 shows why the question has a ZERO-PRODUCT class at all: one 0 digit drags the whole product to 0, permanently. So `product == 0` is not a special case to be guarded, it is a **reliable test** for "N contains a zero", and the classification uses it as one. That is the question rewarding you for noticing what multiplication already knows.

CODE · QUESTION SIX, PART B · VERIFIED ON THE JVM

```
import java.util.Scanner; public class LibraryFines {    // ii. one book's fine, with the ru
```

Output, for 6 books with one invalid entry Book 1: 0 is not a positive number of days. Enter

```

Book 1: Student, 2 day(s) late, fine 0.00
Book 2: Student, 5 day(s) late, fine 40.00
Book 3: Student, 20 day(s) late, fine 200.00
Book 4: Public, 3 day(s) late, fine 150.00
Book 5: Public, 12 day(s) late, fine 430.00
Book 6: Staff, 30 day(s) late, fine 500.00

```

SUMMARY

Total fines: 1320.00

SHORT (1-3): 2, MEDIUM (4-10): 1, EXTENDED (more than 10): 3

Six books, and every rule fires exactly once. Book 1: inside the free 3 days, so 0.00. Book 2: $(5 - 3) \times 20 = 40.00$. Book 3: $(20 - 3) \times 20 = 340$, **capped** to 200.00. Book 4: inside the flat 5 days, 150.00. Book 5: $150 + (12 - 5) \times 40 = 430.00$. Book 6: "Staff" is not "Student", so it is charged as Public, $150 + 25 \times 40 = 1150$, **capped** to 500.00. Both caps, both free periods, the fallback, and the re-prompt: if your test data cannot show all of that, it cannot tell you your program works.

MUST MEMORISE · RETURNING THREE NUMBERS WITHOUT A COLLECTION CLASS

```
return new int[] {shortBand, mediumBand, extendedBand};
```

A method returns **one** thing. When the question wants three and bans `ArrayList`, that one thing is a small array.

The alternatives are worse and the marker knows all of them: three separate methods each looping the array again, which is three passes for one question that said one method; or three static fields quietly written as a side effect, which works and is unreadable.

MEMORY HOOK: one return, so box them. `new int[] {a, b, c}` is a whole array written inline, with no separate declaration and no `new int[3]` followed by three assignments.

TRAP · PARALLEL ARRAYS MUST BE FILLED IN THE SAME ITERATION, OR THE PAIRING ROTs

`daysLate[i]` and `memberType[i]` are the same book, and the only thing making that true is that they are written in the same turn of the same loop. Fill them in two separate loops, or let the re-prompt loop advance `i`, and index 3's days belong to index 4's member type. Nothing throws. Nothing warns. Every fine is simply wrong.

Look at where `i` changes in the answer: **only** in the outer `for`. The validation `while` re-reads days and never touches `i`, which is precisely why book 1's rejected 0 does not cost book 1 its place.

This is why the paper bans `ArrayList` and asks for parallel arrays: not because collections are bad, but because holding two arrays in step by hand is the skill it wants to see you have.

RECALL AND CHECK

Book 6's member type is "Staff". The fine method has no `Staff` branch anywhere. Why does it charge 500.00 rather than crash, or charge nothing?

ANSWER Because the method asks only one question, `memberType.equals("Student")`, and everything that is not `Student` falls into the `else`, which is the `Public` rule. There is no list of valid types to fail against. The question demanded exactly this: "any member type that is not exactly `Student` must be treated as `Public`". An `else` with no condition cannot miss a case, which is what makes it the right shape for a catch-all rule, and it is why the answer never needs to know that "Staff" exists.

After you have marked it

Mock Paper A, in the Older Shape

Mock paper

Question One

MODEL ANSWER

Question One

Write your full answer to **Question One** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q1 A · FORMAL PARAMETER, ACTUAL ARGUMENT, RETURN TYPE · 3 MARKS

Model answer.

A **formal parameter** is the variable named in a method's declaration, inside its brackets. It does not hold a value while the method is not running; it is a name waiting for one, and it exists only inside that method. In `public static int cube(int s)`, the formal parameter is `s`.

An **actual argument** is the value written at the **call site**, which is copied into the formal parameter when the call is made. In `cube(side)`, the actual argument is `side`, and what reaches `s` is a **copy** of its value (Unit 5.4).

The **return type** is the type of the value a method hands back, written immediately before the method's name. In `public static int cube(int s)` it is `int`. A method that returns nothing declares `void`. The return type is **not part of the method signature**, which is why two methods cannot differ by return type alone.

Where the marks are: one each. The word doing the work in the first two is **copy**: parameter is the name, argument is the value, and the value is copied in. Say that and both marks are yours.

Q1 B · DEBUG CUBECALCBROKEN · 3 MARKS

Five errors.

1. **Line 2: `main(String[] )` has no parameter name.** Write `main(String[] args)`.
2. **Line 4: no semicolon** after `cube(side)`.
3. **Line 8: `cube(int)` has no parameter name.** Write `cube(int s)`.
4. **Line 9: `s` does not exist.** This is the symptom of error 3, not a separate fault: name the parameter and `s` exists. Say that out loud in your answer. It shows you found the cause rather than the squiggle.
5. **The closing brace is missing** for `cube` and for the class. Only one `}` is present where two are needed.

What the compiler actually said: four errors, and **not** the one about `s`. It reported the two missing names, the missing semicolon, and "reached end of file while parsing" for the brace. Only after those are fixed does it turn to line 9 and report `cannot find symbol: variable s` three times, once per use.

Three marks, five errors, and the fifth is invisible until the third is fixed. Do not stop at three.

CODE · Q1 B CORRECTED · VERIFIED ON THE JVM

```
public class CubeCalc {    public static void main(String[] args) {        int side = 4;

OutputCube is: 64.0
```

64.0, not 64. `cube` returns an `int`, and that `int` is stored into a `double result`. Java **widens** it on the way in (Unit 5.7), so what is printed is a double, and a double always prints its point. Writing "Cube is: 64" throws away a mark that costs nothing.

Q1 C · DRY RUN TALLY WITH 3 AND 5 · 4 MARKS

The answers are 4 and 9. Not 5 and 10.5, which is what the arithmetic gives if you read $(k + 1) / 2$ as a fraction.

Both $k + 1$ and 2 are `int`, so `/` is **integer division** and every term is chopped down to a whole number (Unit 3.6). Nothing rounds: $3/2$ is 1, not 1.5 and not 2.

DRY RUN · TALLY(5), ONE ROW PER ITERATION

k	k + 1	(k + 1) / 2	as real arithmetic	total
1	2	1	1.0	1
2	3	1	1.5	2
3	4	2	2.0	4
4	5	2	2.5	6
5	6	3	3.0	9

tally(3) stops after the third row: 1, 2, 4. **tally(5)** runs all five: 9.

Column four is not part of the answer. It is there to show you the trap: the two columns agree on rows 1, 3 and 5 and disagree on rows 2 and 4, so the loss is exactly 0.5 twice, and the real-arithmetic total of 10.5 becomes 9. A candidate who never notices the division is an `int` division still gets rows 1, 3 and 5 right, which is precisely why this trap survives: it is half invisible.

CODE · Q1 C, THE SAME METHOD WITH A MAIN THAT CALLS IT · VERIFIED ON THE JVM

```
public class TallyTrace {
    public static int tally(int n) {
        int total = 0;
        for (int k = 1; k <= n; k++) {
            total += (k + 1) / 2;
        }
        return total;
    }
}

Outputtally(3) = 4
tally(5) = 9
```

Q1 D · OVERLOAD VOLUME · 7.5 MARKS

Read the three shapes before you write anything. A cube needs one length, a cylinder needs a radius and a height, a box needs three lengths. One parameter, two parameters, three parameters: the three overloads differ by **arity**, so the compiler can always tell them apart.

CODE · QUESTION ONE, PART D · VERIFIED ON THE JVM

```
public class VolumeOverload {
    // i. a cube: one length is all it takes
    public static double volume(int side) {
        return side * side * side;
    }

    // ii. a cylinder: radius and height
    public static double volume(double r, double h) {
        return Math.PI * r * r * h;
    }

    // iii. a box: three lengths
    public static double volume(int l, int w, int h) {
        return l * w * h;
    }
}

OutputCube, side 4: 64.00
Cylinder, r 3, h 5: 141.37
Box, 2 by 3 by 4: 24.00
```

TRAP · THIS QUESTION IS ANSWERABLE. ITS ANCESTOR WAS NOT

Compare this with the 2024/25 paper's Question One D, solved in S.9. That one asked for area overloaded for a circle, a rectangle and a triangle. A circle takes one length, but a rectangle takes two and a triangle takes two, so two of the three overloads have the **same signature**, `area(double, double)`, and the compiler rejects the class outright with "method `area(double,double)` is already defined". **That question, as printed, cannot be answered as printed.**

`volume` escapes because 1, 2 and 3 lengths are three different counts. That is luck, not design, and it is worth understanding rather than enjoying: **overloading resolves on the parameter list alone**. Not on the names you give the parameters, not on what you meant, and not on the return type. Two

shapes that need the same numbers cannot be overloads, and the fix is to name them `rectangleArea` and `triangleArea`, or to make the lists genuinely differ.

RECALL AND CHECK

Why can `volume(double side)` and `volume(double radius, double height)` coexist, when `area(double, double)` twice cannot?

ANSWER Because the signature is the name plus the **parameter list**, and one parameter is a different list from two. The two area methods have identical lists, so they have identical signatures, so they are the same method declared twice. What the parameters *mean* to you is not in the signature.

Question Two

MODEL ANSWER

Question Two

Write your full answer to **Question Two** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q2 A · DECLARATION, CREATION, INDEX, PARALLEL ARRAYS · 3 MARKS

Model answer.

Array declaration announces a name and its type: `int[] a;`. It creates **no array**. It creates a reference variable capable of referring to one, and until something is assigned to it the variable refers to `null`.

Array creation is what actually makes the array and reserves its elements: `a = new int[4];`, or the shorthand `int[] a = {1, 3, 5, 7};` which declares and creates in one line. The length is fixed at the moment of creation and can never change afterwards.

The **index** is the position of an element, counted **from 0**. In an array of length 4 the valid indexes are 0, 1, 2 and 3, so the last index is always `length - 1` and `a[a.length]` is always wrong.

Parallel arrays are two or more arrays of the same length in which the same index refers to the same subject: `names[2]` and `ages[2]` are the name and the age of one person. They are the way this course models a record without a class and without a collection.

Where the marks are: the distinction in the first two is the whole point. "Declaration names it, creation makes it." A declaration that is never followed by a creation is exactly the bug in part B.

Q2 B · DEBUG REVERSEARRAYBROKEN · 3 MARKS

Four errors. The compiler reports one.

1. **Line 5: `i` is not an update.** The third slot of a for must be a statement; `i` on its own is an expression that does nothing. Write `i++`. This is the only error javac reports: "not a statement".
2. **Line 4: `int b`; declares an int, not an array,** and never creates anything. Write `int[] b = new int[a.length];`.

3. **Line 5: `i <= a.length` is off by one.** Write `i < a.length`.
4. **Line 6: `a[a.length - i]` is out of bounds** the moment `i` is 0, because that is `a[4]` in an array of length 4. Reversing needs `a[a.length - 1 - i]`.
5. **Line 8: `i` does not exist here.** It was declared inside the `for`, so it dies with it. Printing the array needs its own loop.

What the compiler actually said: one error, "not a statement". Fix that alone and three more appear: "array required, but int found" twice, and "cannot find symbol: variable i". Error 4 is never reported by the compiler at all, because it is not a compile error: it is an `ArrayIndexOutOfBoundsException` that waits until the program is running, and the message it eventually gives is `Index 4 out of bounds for length 4`.

One reported error. Four real ones. This snippet is the whole argument of MS.1 in nine lines.

CODE · Q2 B CORRECTED · VERIFIED ON THE JVM

```
public class ReverseArray {    public static void main(String[] args) {        int[] a = {1,
```

Output7

```
5
3
1
```

Q2 C · DRY RUN MAXORDROP · 4 MARKS

The question asks for the output **of each iteration** and the final output, so a bare "8" is not the answer even though 8 is the final value. Write the four lines.

DRY RUN · MAXORDROP, ONE ROW PER ITERATION

i	data[i]	data[i] > x?	what happens	x after
<i>before the loop:</i> x = data[0]				5
1	3	3 > 5? no	x = x - 3	2
2	9	9 > 2? yes	x = 9	9
3	1	1 > 9? no	x = x - 1	8

The `else` is what makes this worth tracing. The variable is not a running maximum: when the new value does not beat it, it is **subtracted from it**, so `x` can fall as well as rise, and the final 8 is not any element of the array.

CODE · Q2 C, RUN FOR REAL · VERIFIED ON THE JVM

```
public class MaxOrDrop {    public static void main(String[] args) {        int[] data = {5,
```

Outputi = 1, x = 2

i = 2, x = 9

i = 3, x = 8

Final x = 8

CODE · QUESTION TWO, PART D · VERIFIED ON THE JVM

```
import java.util.Scanner;public class ArrayReport {    // ii a. find, count and display the
```

Output, for the inputs 12, 25, -4, 30, 7, 15, 3, 20
Multiples of 5: 25, 30, 15, 20
How many: 4
Sum of the rest: 18
Minimum: -4

Input is a Scanner here so the run can be printed on a page. In the hall, each prompt is JOptionPane.showInputDialog and each display is showMessageDialog (F.11). Everything else is unchanged.

TRAP · "-4 % 5" IS NOT WHAT YOU EXPECT, AND HERE IT DOES NOT MATTER

-4 % 5 in Java is **-4**, not 1: the remainder takes the sign of the **left** operand (Unit 3.6). So the test values[i] % 5 == 0 still correctly rejects -4, because -4 is not 0 either way.

But had the question asked for **odd** numbers and you had written values[i] % 2 == 1, every negative odd number would have failed the test, because -7 % 2 is **-1**. That is why this book always writes % 2 != 0 for odd. It is right for negatives; the other is not.

Question Three

MODEL ANSWER

Question Three

Write your full answer to **Question Three** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q3 A · FILEWRITER, BUFFEREDREADER, CHECKED EXCEPTION · 3 MARKS

Model answer.

FileWriter is a class in java.io that writes characters to a file. Constructing one, new FileWriter("log.txt"), creates the file if it does not exist and **truncates it if it does**; the two-argument form new FileWriter("log.txt", true) appends instead. It writes characters one at a time, which is why it is usually wrapped in something more convenient, such as PrintWriter or BufferedWriter.

BufferedReader is a class in java.io that wraps another reader and reads it in blocks rather than a character at a time. It is worth naming for one method: readLine(), which returns the next line as a String, or null at end of file. That null is the loop's stopping condition.

A **checked exception** is one the compiler **insists** you deal with: you must either catch it or declare it with throws, and if you do neither the program does not compile. IOException is the example that matters here. The reasoning is that a missing file is not a programming mistake, it is a fact about the world, so the compiler forces you to have a plan for it. An **unchecked** exception, such as ArrayIndexOutOfBoundsException, is not policed this way, because it is a bug rather than a circumstance.

Where the marks are: for the third, the phrase "caught or declared, or it will not compile" is the mark. Part B is that sentence with the compiler saying it out loud.

Q3 B · DEBUG NOTEWRIERBROKEN · 3 MARKS

Four errors, in two families. The compiler reports one.

1. **Line 5: `FileWriter("log.txt")` has no `new`.** As written, that is a call to a method named `FileWriter`, which does not exist. A constructor is only a constructor when `new` is in front of it. This is the only error javac reports: "cannot find symbol: method `FileWriter(String)`".
2. **Lines 5, 6 and 7 all throw `IOException`, which is checked**, and nothing here catches or declares it. Three errors, one cause. Wrap the work in `try (FileWriter out = new FileWriter("log.txt")) { ... } catch (IOException e) { ... }`, or declare `throws IOException` on `main`.
3. **The file is never closed on the failure path.** Even with a catch, a plain `out.close()` at the end is skipped if `write` throws. **`try-with-resources`** closes it either way, which is exactly why it exists.

What the compiler actually said: one error, about `new`. The three unreported `IOException` errors are invisible until it is fixed, because until then the compiler does not know what type `FileWriter("log.txt")` has, so it cannot know what that line throws.

This is the single best illustration on the paper of MS.1's rule. The **interesting** error, the one the question is really about, is the one the compiler will not tell you about until you have fixed the boring one.

CODE · Q3 B CORRECTED · VERIFIED ON THE JVM

```
import java.io.FileWriter;import java.io.IOException;public class NoteWriter {    public static void main(String[] args) {        try {            FileWriter out = new FileWriter("log.txt");            out.write("log.txt written");            out.close();        } catch (IOException e) {            e.printStackTrace();        }    }}
```

Q3 C · WHAT LANDS IN REPORT.TXT · 4 MARKS

The answer, three lines:

```
report.txtUnit: 7
Rate: 12.3
Total:  3.14
```

Four things are being tested here at once, and each is worth a mark.

1. **`print` then `println`.** `print("Unit: ")` adds no line break, so the 7 lands on the same line and **then** the line ends. One line, not two.
2. **`%.1f` of 12.345 is 12.3.** One decimal place means the 3 stays and everything after it decides whether it moves. What follows is 45, and 0.045 is less than half of 0.1, so it does not. See the box below, because the reason this is *ordinary* is itself worth a mark.
3. **`%.1f` ends the line** after the rate.
4. **`%.2f` of 3.14159 is 3.14:** two decimals, and then **padded to a total width of 5**, which is 4 characters wide, so one space goes in front. The line reads `Total:` , a space from the literal, then a **second** space from the padding.

And there is no line break after the last one, because `printf("Total: %.2f", ...)` has no `%.1f`. The file ends without a newline.

MUST MEMORISE · `PRINTF` ROUNDS THE DIGITS YOU WROTE, HALF UP. YOU CAN TRUST YOUR PAPER ARITHMETIC

To answer any printf rounding question: **take the literal as printed, round it half up to the places asked for, and write that down.** That is the answer. Every time.

You may have been told the frightening version: that double is inexact, so %.2f of 1.005 comes out as 1.00 because 1.005 is really stored as 1.00499999999999989. The first half is true. **The conclusion is false in Java**, and it is worth knowing why, because it is true in some other languages and the folklore travels.

Java's printf does not round the raw binary value. It first takes the **shortest decimal that uniquely identifies** that double, which for the literal 1.005 is the string 1.005, and rounds **that**, half up. So it prints **1.01**, exactly as your paper arithmetic says.

This was measured, not assumed. Every three-decimal value from 0.001 to 99.999 was formatted with %.2f on a real JVM and compared against both rules:

Rule	Values tested	Disagreements
round the digits you wrote, half up	99,999	0
round the stored binary value, half up	99,999	4,800

Zero. The scary rule is the one that is wrong, and it is wrong about one value in twenty: at 0.015, printf prints **0.02** while the stored value sits just below 0.015 and the folklore predicts 0.01.

MEMORY HOOK: printf reads what you wrote, not what the hardware kept. Unit 3.7's warning about inexact doubles is about **arithmetic**, where the error accumulates and is real. It is not about displaying a literal.

TRAP · THE INEXACTNESS IS REAL. IT JUST SHOWS UP SOMEWHERE ELSE

Do not read the box above as "doubles are fine". They are not, and Unit 3.7 means what it says. The difference is **where** the damage lands.

Displaying a literal is safe, because printf rounds the decimal you wrote. **Comparing** and **accumulating** are not: $0.1 + 0.2 == 0.3$ is false, and no formatting rule saves you from that, because the damage happened in the addition, before printf ever saw it.

So the rule has a boundary, and it is this: trust printf to show a value faithfully; never trust `==` between two doubles.

CODE · Q3 C, THE SAME PROGRAM, READING ITS OWN FILE BACK · VERIFIED ON THE JVM

```
import java.io.*;public class ReportWriter {    public static void main(String[] args) thro
```

Output, with every space shown as a dotline 1: Unit:.7

line 2: Rate:.12.3

line 3: Total:..3.14

The read-back is not part of the answer; it is how this book proves the answer instead of asserting it. Count the dots on line 3: **two**. One is the space you typed inside "Total: " and the other is `%.2f` padding 3.14 out to five columns. On line 2, **one** dot, and 12.3 after it.

CODE · QUESTION THREE, PART D · VERIFIED ON THE JVM

```
import java.io.BufferedReader;import java.io.FileReader;import java.io.FileWriter;import java
books.txt after one run, every space shown as a dotTitle.....Copies..Price....
Algebra....12.....2500.00..
Optics....4.....1875.50..
Ecology....7.....990.25...
```

The dots are this book showing you the padding; the file has spaces. Read the alignment: `%-10s` puts the title in ten columns, left-aligned, and `%-9.2f` puts the price in nine, so the columns line up whatever the values are.

MUST MEMORISE · APPEND LIVES ON THE FILEWRITER, NOT THE PRINTWRITER

`new PrintWriter("books.txt")` **truncates**. There is no append flag on that constructor.

`new PrintWriter(new FileWriter("books.txt", true))` **appends**.

Part iv of this question says "ensure the file is appended to if it already exists", and it is the whole reason the two classes are stacked. The `true` belongs to the `FileWriter` underneath; `PrintWriter` is only there to give you `printf`.

MEMORY HOOK: the flag is on the writer that touches the disk. `PrintWriter` never touches the disk; it formats and hands down. Ask "who opens the file?" and the answer tells you where `true` goes.

Question Four

MODEL ANSWER

Question Four

Write your full answer to **Question Four** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q4 A · COUNTER CONTROLLED, SENTINEL CONTROLLED, CONTINUE · 3 MARKS

Model answer.

A **counter-controlled** loop repeats a number of times that is known before the loop starts. It counts. A `for` loop is its natural shape, because the start, the test and the step sit together on one line: `for (int i = 0; i < 8; i++)` runs exactly 8 times. It is also called a **definite** loop.

A **sentinel-controlled** loop repeats until a particular value arrives to say stop. That value is the **sentinel**, and it is not data: it is a signal, chosen so it cannot be mistaken for a real reading. Reading lines until `readLine()` returns `null` is sentinel-controlled, and so is "keep asking until the user types -1". The number of repetitions is not known in advance, which is why a `while` is its natural shape. It is also called an **indefinite** loop.

continue abandons the **rest of the current iteration** and goes straight to the next one. It does not leave the loop: that is `break`. In a `for` loop the update still runs, so `continue` is safe there; in a `while` loop it jumps straight back to the test, so if the increment sits below it, the loop never moves again and hangs (Unit 4.7).

Where the marks are: "known in advance" against "known only when it happens" carries the first two. For the third, the distinction from break is the mark: **continue skips an iteration, break leaves the loop**.

Q4 B · DEBUG COUNTUPBROKEN · 3 MARKS

Four errors, and the worst one compiles.

1. **Line 3: no semicolon** after `int count = 1`.
2. **Line 4: `float output = ""`; is the wrong type.** A float is a number and "" is text; there is no conversion between them. It must be `String output = ""`;
3. **Line 6: `while (count <= 5)`; has a stray semicolon.** That semicolon **is** the loop body: an empty statement. The block below it then runs once, on its own, as an ordinary block. Nothing inside the empty body ever changes count, so the test stays true forever and the program **hangs**. Delete the semicolon.
4. **Line 8: Count is not a statement**, and it is not count either, because Java is case sensitive. It should be `count++`;

What the compiler actually said: three errors: the missing semicolon, and Count twice ("not a statement" and ";" expected"). Fix those and **two more** appear, both incompatible types: String cannot be converted to float. And error 3, the stray semicolon, is **never reported at all**. It was compiled on its own to check: it is legal Java. It is an infinite loop, and the compiler has no opinion about those.

This snippet is the clearest case on the paper: **the most damaging bug in it is the one the compiler is happiest with**.

CODE · Q4 B CORRECTED · VERIFIED ON THE JVM

```
public class CountUp {    public static void main(String[] args) {        int count = 1;
```

Output1 2 3 4 5

There is a trailing space after the 5, because the loop appends `count + " "` every time round, including the last. The question asked for 1 2 3 4 5 and this is what that instruction produces. Do not invent a fix the question did not ask for.

Q4 C · STATE THE OUTPUT OF EACH ITERATION AND THE FINAL SUM · 4 MARKS

Three lines, then the total. The loop runs five times and prints three, because continue jumps over the print.

DRY RUN · SKIPTOTAL, ONE ROW PER ITERATION

i	inputs[i]	negative?	what happens	printed
0	4	no	sum = 0 + 4 = 4	i = 0, sum = 4
1	-2	yes	continue: skip the rest	nothing
2	6	no	sum = 4 + 6 = 10	i = 2, sum = 10
3	-8	yes	continue: skip the rest	nothing

4	5	no	sum = 10 + 5 = 15	i = 4, sum = 15
---	---	----	-------------------	-----------------

Look at the **i** values that print: **0, 2, 4**. The gaps are the evidence. A candidate who writes three lines numbered 0, 1, 2 has read `continue` as "do not add" rather than "skip the rest of this turn", and loses a mark on a detail they never looked at.

CODE · Q4 C, RUN FOR REAL · VERIFIED ON THE JVM

```
public class SkipTotal {    public static void main(String[] args) {        int sum = 0;

Outputi = 0, sum = 4
i = 2, sum = 10
i = 4, sum = 15
Sum = 15
```

TRAP · CONTINUE HERE, BREAK ON THE REAL PAPER, AND THE KEYWORD IS THE WHOLE QUESTION

The 2024/25 paper's Question Four C is this question with `break` instead of `continue`, over the array {3, 4, -1, 2}. It answers **7**: the loop **stops** at -1, so the 2 sitting behind it is never reached and is silently missing from a sum that looks complete.

Here, `continue` only skips the negatives and carries on, so the 5 behind -8 **is** counted and the answer is 15. Same array shape, same question shape, one keyword, and a different answer.

Both have been asked. When you meet either keyword in a dry-run question, say which one it is out loud before you trace a single row.

CODE · QUESTION FOUR, PART D · VERIFIED ON THE JVM

```
import java.util.Scanner;public class SpeedCheck {    // i. the state: one data field    pr:

Output, for the input 75Normal
```

Input is a Scanner here so the run can be printed on a page. In the hall it is
`Double.parseDouble(JOptionPane.showInputDialog("Enter the speed:"))` and the
verdict goes out through `showMessageDialog(F.11)`.

MUST MEMORISE · "BETWEEN 40 AND 100 INCLUSIVE" IS ONE ELSE-IF, NOT TWO TESTS

```
if (speed < 40) ... else if (speed <= 100) ... else ...
```

The middle branch needs **no** lower bound. Reaching it already proves `speed >= 40`, because the first branch did not fire.

Writing `else if (speed >= 40 && speed <= 100)` is not wrong and costs only ink. But the same misunderstanding written as three separate `if` statements **is** wrong, because then nothing guarantees anything and every test runs on every value. Unit 4.4 is the paper's own obsession for this reason.

MEMORY HOOK: an else-if arrives already knowing what failed. Its job is only the upper bound. Every branch you reach carries the previous conditions' failure behind it, for free.

CODE · Q4 D, EVERY BOUNDARY PROBED · VERIFIED ON THE JVM

```
public class SpeedBench {    private double speed;    public SpeedBench(double speed) {
```

```
Output39.9 -> Too Slow
40.0 -> Normal
75.0 -> Normal
100.0 -> Normal
100.1 -> Too Fast
```

Not part of the answer: this is how you check an if-else ladder. Both edges of "inclusive" are probed, 40.0 and 100.0, and both land in **Normal**. Test the boundaries, never the middle: 75 would have passed even with < written where <= belongs, and told you nothing.

Question Five

MODEL ANSWER

Question Five

Write your full answer to **Question Five** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

Q5 A · SUPER, @OVERRIDE, RUNTIME POLYMORPHISM · 3 MARKS

Model answer.

super is a keyword that refers to the superclass part of the current object. It has two uses.

`super(...)`, as the **first statement** of a constructor, calls the superclass constructor, which is how a subclass gets the inherited fields initialised. `super.method()` calls the superclass version of a method that the subclass has overridden, which is how an override extends the original rather than replacing it.

@Override is an annotation placed above a method to state that it is meant to override an inherited one. It changes nothing at run time. Its whole value is that the **compiler checks the claim**: if the method does not actually override anything, because the name or the parameter list does not match, it is a compile error instead of a silent new method that never gets called.

Runtime polymorphism, also called dynamic method dispatch, is this: the method that runs is chosen by the **object's actual class at run time**, not by the type of the reference variable at compile time. So an Employee reference holding a Manager runs Manager's `displayInfo()`. It is what makes overriding useful, and it is the opposite of overloading, which is settled entirely at compile time (Unit 5.6).

Where the marks are: for the third, the sentence that earns it is "the **object** decides, not the reference". For the second, "the compiler checks it" is the mark; "it makes it override" is wrong, and that is the commonest answer.

CODE · QUESTION FIVE, PART B · VERIFIED ON THE JVM

The question asks for the method definition only, so this is the whole answer:

```
public static int countAbove(double[] values, double limit) {    int count = 0;    for (int
```

Three marks and four decisions: the **return type** is `int`, because it returns a count; the **parameters** are the array and the limit; the loop runs `i < values.length`; and the test is `>`, because the question said **strictly** greater. Write `>=` and you have answered a different question.

CODE · Q5 B, THE SAME METHOD WITH A MAIN AROUND IT · VERIFIED ON THE JVM

```
public class CountAboveDemo {    public static int countAbove(double[] values, double limit) {  
    }  
}  
  
Outputabove 70.0: 1  
above 39.5: 4  
above 99.0: 0
```

Not part of the answer, but read the first line: the array holds `70.0` twice and "above 70.0" is **1**, not 3. That is `>` doing exactly what was asked. The third line proves the method survives a limit nothing beats.

Q5 C · IDENTIFY THE ERRORS, AND STATE THE OUTPUT AFTER CORRECTION · 4 MARKS

Five errors, and one of them prints the wrong answer without ever failing.

1. **Line 3: `int values = {4, 8, 12, 16};` is not an array.** Write `int[] values`. Everything else on the list is downstream of this one.
2. **Line 4: `values[3]` is not the third value.** Indexes count from 0, so the third value is `values[2]`, which is **12**. `values[3]` is 16, the fourth. This is a **logic** error: it compiles, it runs, and it prints a number that is simply wrong.
3. **Line 6: `i <= values.length` is off by one** and would throw `ArrayIndexOutOfBoundsException` at `values[4]`. Write `i < values.length`.
4. **Line 6: `i` is not an update.** Write `i++`.
5. **Line 9: `"Sum is: " + )` is missing its operand.** Write `"Sum is: " + sum`.

What the compiler actually said: two errors, "not a statement" and "illegal start of expression". Fix those and **four more** appear, all downstream of error 1: "illegal initializer for int", "array required, but int found" twice, and "int cannot be dereferenced" for `values.length`. And error 2 is **never reported at any stage**, because printing the wrong element is not illegal. It is just wrong.

The output after correction:

```
OutputThird value is: 12  
Sum is: 40
```

$4 + 8 + 12 + 16 = 40$. If you answered "Third value is: 16" you corrected the compiler's errors and left the examiner's error in place, which is the one this question is actually about.

CODE · Q5 C CORRECTED · VERIFIED ON THE JVM

```
public class Totals {    public static void main(String[] args) {        int[] values = {4,  
    }  
}  
  
OutputThird value is: 12  
Sum is: 40
```

CODE · QUESTION FIVE, PART D · VERIFIED ON THE JVM

```
// i. the superclassclass Employee {    protected int salary;    // i b. a constructor that
```

OutputEmployee salary: N80000 per month

Manager salary: N250000 per month. This post carries a leadership allowance.

Employee salary: N80000 per month

Manager salary: N250000 per month. This post carries a leadership allowance.

The last two lines are the point. They come from an Employee[] array, so at compile time both slots are just Employee. At run time each object runs **its own** displayInfo(). That is polymorphism, and the array is the shortest honest way to show it: one call site, two behaviours, chosen by the object.

TRAP · PROTECTED, NOT PRIVATE, OR THE SUBCLASS CANNOT SEE THE FIELD

salary is declared protected, and that is deliberate. Declare it private and Manager's displayInfo() **will not compile**: "salary has private access in Employee". A subclass inherits a private field in the sense that the object carries it, but it **cannot name it**.

Both ways are defensible in real code, and private plus a getter is the better habit, which is exactly what the 2025/26 Question Four demands. Here the question said "an instance variable salary" and the subclass must print it, so protected is the shortest correct reading. Say which you chose and why, in a comment. That is a mark for one line.

Question Six

MODEL ANSWER

Question Six

Write your full answer to **Question Six** in your book first, one clear point for every mark. Then open the model answer and mark yourself against it.

CODE · QUESTION SIX, PART A · VERIFIED ON THE JVM

```
public class LoopDemo {    public static void main(String[] args) {        // i. a for loop
```

Output2 4 6 8 10

9 8 7 6 5 4 3 2 1

100

One mark each, and each one is a different loop **because of what it knows**. The for counts and steps by 2. The while counts down, and its n-- must be inside the body or it never ends. The do-while is the only loop that runs its body **before** its test, which is exactly why it can print once with a condition that is false from the very start.

MUST MEMORISE · DO-WHILE RUNS FIRST AND ASKS AFTERWARDS

do { ... } while (false); runs its body **exactly once**.

while (false) { ... } runs its body **exactly zero times**.

That single difference is the entire reason do-while exists, and "print only the number 10 once" is a question built to test it. Note the **semicolon** after while (false): a do-while is the one loop that

ends in one, and leaving it off is a compile error.

MEMORY HOOK: do first, ask later. A do-while is a shop that lets you in and then checks your ticket. You are always getting in at least once.

Q6 B · THE RETURN VALUE OF COMPUTE(2736) · 3 MARKS

The answer is 9. The method walks the digits from the right using `% 10` to take one off and `/ 10` to drop it, and adds up only the digits that divide by 3.

The question says "show the steps", so the table **is** the answer. Two of the three marks are the working.

DRY RUN · COMPUTE(2736)

x	d = x % 10	d % 3 == 0?	result	x = x / 10
2736	6	yes	0 + 6 = 6	273
273	3	yes	6 + 3 = 9	27
27	7	no	9	2
2	2	no	9	0
0	x > 0 is false: the loop ends and 9 is returned			

Watch the last row. `2 / 10` is **0**, not 0.2: integer division is what terminates this loop, and every digit-walking loop in this course ends the same way (Unit 3.6). If `/` gave 0.2 the loop would never stop.

CODE · Q6 B, RUN FOR REAL · VERIFIED ON THE JVM

```
public class ComputeTrace {    public static int compute(int x) {        int result = 0;

Outputcompute(2736) = 9
```

Q6 C · DEBUG ODDFINDERBROKEN, AND STATE THE CORRECT OUTPUT · 4 MARKS

Five errors, and the first one compiles perfectly.

- Line 2: `main(String args)` is not a main method.** It needs `String[] args`. This is the one to notice. It is **legal Java**: it compiles without a murmur, and it is simply an ordinary method that happens to be called main. The failure waits until you run it, and the JVM says `Error: Main method not found in class OddFinderBroken`. It was compiled and run to confirm exactly that.
- Line 3: no semicolon** after `printOdd(9.0)`.
- Line 3: `9.0` is a double** and the method takes an `int`. Java will not narrow it for you: "incompatible types: possible lossy conversion from double to int". Write `printOdd(9)`.
- Line 6: `printOdd(int)` has no parameter name.** Write `printOdd(int n)`.
- Line 7: `for (...);` has a stray semicolon.** The loop body is that empty statement, so the loop spins 9 times doing nothing, and then the `if` below runs **once**, outside the loop, where `i` no longer exists.

What the compiler actually said: two errors, the semicolon and the missing parameter name. Fix those and **three more** appear: the lossy conversion, and `cannot find symbol: variable i`

twice, which is error 5 finally surfacing. Error 1 is never reported at any stage.

The output after correction:

Output1 3 5 7 9

CODE · Q6 C CORRECTED · VERIFIED ON THE JVM

```
public class OddFinder {    public static void main(String[] args) {        printOdd(9);
```

Output1 3 5 7 9

MUST MEMORISE · A STRAY SEMICOLON AFTER FOR OR WHILE IS THE BODY

for (int i = 1; i <= n; i++); is a **complete loop** with an empty body. The indented block below it is **not** inside it.

It appears twice on this mock, in Q4 B and Q6 C, and once on the real 2024/25 paper, in its Question Four B and Question Six C. That is not a coincidence: this lecturer plants it deliberately, because it is the purest example of code that compiles and lies.

Java does not care about your indentation. It cares about the semicolon, and a semicolon where a body should be **is** a body: an empty one.

MEMORY HOOK: a loop header never ends in a semicolon. If you see); at the end of a for or while line, that loop is already finished and everything you thought was inside it is not.

CODE · QUESTION SIX, PART D · VERIFIED ON THE JVM

```
public class AscendingDigits {    // i, ii. true only if the digits strictly increase from :
```

Output1359 -> true

1354 -> false

1223 -> false

7 -> true

The method walks right to left, so it compares each digit with the one **to its right** and demands digit < previous. Reading right to left and testing "smaller" **is** reading left to right and testing "larger": the same fact, from the end that % 10 can actually reach.

TRAP · "STRICTLY" IS A WHOLE MARK, AND 1223 IS THE TEST FOR IT

1223 answers **false**, and the digits never go down. They stop going **up**: 2 is not **strictly** less than 2. Write digit > previous instead of digit >= previous and 1223 wrongly answers true, while every other example on the paper still passes. That is one character, one example, one mark.

The single digit 7 is the other test worth writing: the loop body never runs at all, and the method must still answer **true**. A method that cannot survive a one-digit input is a method that was never traced.

Choose your own test cases the way this book chose these four: one that passes, one that fails on a decrease, one that fails on a tie, and one edge. Two cases were asked for. Four cost you a minute and prove you thought.

RECALL AND CHECK

Of the six debug snippets on this paper, three contain a bug that the compiler never reports at any stage. Name the three bugs, and say what each would do to a running program.

ANSWER Q4 B: the stray semicolon after `while (count <= 5)` makes an empty body, so the program hangs forever. **Q5 C:** `values[3]` where `values[2]` was meant, so it prints 16 and calls it the third value: it runs perfectly and answers wrongly. **Q6 C:** `main(String args)` compiles as an ordinary method, so the JVM finds no main method and the program cannot start at all.

One hangs, one lies, one never starts. None of them is a compile error, and that is the exam's whole point.

After you have marked it
